

CHAPTER

8

Sequence Labeling for Parts of Speech and Named Entities

To each word a warbling note

A Midsummer Night's Dream, V.I

parts of speech

Dionysius Thrax of Alexandria (c. 100 B.C.), or perhaps someone else (it was a long time ago), wrote a grammatical sketch of Greek (a “*technē*”) that summarized the linguistic knowledge of his day. This work is the source of an astonishing proportion of modern linguistic vocabulary, including the words *syntax*, *diphthong*, *clitic*, and *analogy*. Also included are a description of eight **parts of speech**: noun, verb, pronoun, preposition, adverb, conjunction, participle, and article. Although earlier scholars (including Aristotle as well as the Stoics) had their own lists of parts of speech, it was Thrax’s set of eight that became the basis for descriptions of European languages for the next 2000 years. (All the way to the *Schoolhouse Rock* educational television shows of our childhood, which had songs about 8 parts of speech, like the late great Bob Dorough’s *Conjunction Junction*.) The durability of parts of speech through two millennia speaks to their centrality in models of human language.

named entity

Proper names are another important and anciently studied linguistic category. While parts of speech are generally assigned to individual words or morphemes, a proper name is often an entire multiword phrase, like the name “Marie Curie”, the location “New York City”, or the organization “Stanford University”. We’ll use the term **named entity** for, roughly speaking, anything that can be referred to with a proper name: a person, a location, an organization, although as we’ll see the term is commonly extended to include things that aren’t entities per se.

POS

Parts of speech (also known as **POS**) and named entities are useful clues to sentence structure and meaning. Knowing whether a word is a noun or a verb tells us about likely neighboring words (nouns in English are preceded by determiners and adjectives, verbs by nouns) and syntactic structure (verbs have dependency links to nouns), making part-of-speech tagging a key aspect of parsing. Knowing if a named entity like *Washington* is a name of a person, a place, or a university is important to many natural language processing tasks like question answering, stance detection, or information extraction.

In this chapter we’ll introduce the task of **part-of-speech tagging**, taking a sequence of words and assigning each word a part of speech like NOUN or VERB, and the task of **named entity recognition (NER)**, assigning words or phrases tags like PERSON, LOCATION, or ORGANIZATION.

sequence labeling

Such tasks in which we assign, to each word x_i in an input word sequence, a label y_i , so that the output sequence Y has the same length as the input sequence X are called **sequence labeling** tasks. We’ll introduce classic sequence labeling algorithms, one generative—the Hidden Markov Model (HMM)—and one discriminative—the Conditional Random Field (CRF). In following chapters we’ll introduce modern sequence labelers based on RNNs and Transformers.

8.1 (Mostly) English Word Classes

Until now we have been using part-of-speech terms like **noun** and **verb** rather freely. In this section we give more complete definitions. While word classes do have semantic tendencies—adjectives, for example, often describe *properties* and nouns *people*—parts of speech are defined instead based on their grammatical relationship with neighboring words or the morphological properties about their affixes.

	Tag	Description	Example
Open Class	ADJ	Adjective: noun modifiers describing properties	<i>red, young, awesome</i>
	ADV	Adverb: verb modifiers of time, place, manner	<i>very, slowly, home, yesterday</i>
	NOUN	words for persons, places, things, etc.	<i>algorithm, cat, mango, beauty</i>
	VERB	words for actions and processes	<i>draw, provide, go</i>
	PROPN	Proper noun: name of a person, organization, place, etc..	<i>Regina, IBM, Colorado</i>
Closed Class Words	INTJ	Interjection: exclamation, greeting, yes/no response, etc.	<i>oh, um, yes, hello</i>
	ADP	Adposition (Preposition/Postposition): marks a noun's spacial, temporal, or other relation	<i>in, on, by, under</i>
	AUX	Auxiliary: helping verb marking tense, aspect, mood, etc.,	<i>can, may, should, are</i>
	CCONJ	Coordinating Conjunction: joins two phrases/clauses	<i>and, or, but</i>
	DET	Determiner: marks noun phrase properties	<i>a, an, the, this</i>
	NUM	Numeral	<i>one, two, first, second</i>
	PART	Particle: a function word that must be associated with another word	<i>'s, not, (infinitive) to</i>
	PRON	Pronoun: a shorthand for referring to an entity or event	<i>she, who, I, others</i>
Other	SCONJ	Subordinating Conjunction: joins a main clause with a subordinate clause such as a sentential complement	<i>that, which</i>
	PUNCT	Punctuation	<i>, , ()</i>
	SYM	Symbols like \$ or emoji	<i>\$, %</i>
	X	Other	<i>asdf, qwfg</i>

Figure 8.1 The 17 parts of speech in the Universal Dependencies tagset (de Marneffe et al., 2021). Features can be added to make finer-grained distinctions (with properties like number, case, definiteness, and so on).

closed class Parts of speech fall into two broad categories: **closed class** and **open class**.
open class Closed classes are those with relatively fixed membership, such as prepositions—

function word new prepositions are rarely coined. By contrast, nouns and verbs are open classes—
 Closed class words are generally **function words** like *of, it, and, or you*, which tend

to be very short, occur frequently, and often have structuring uses in grammar.
 Four major open classes occur in the languages of the world: **nouns** (including proper nouns), **verbs**, **adjectives**, and **adverbs**, as well as the smaller open class of **interjections**. English has all five, although not every language does.

noun **Nouns** are words for people, places, or things, but include others as well. **Common noun** **Common nouns** include concrete terms like *cat* and *mango*, abstractions like *algorithm* and *beauty*, and verb-like terms like *pacing* as in *His pacing to and fro became quite annoying*. Nouns in English can occur with determiners (*a goat, this bandwidth*) take possessives (*IBM's annual revenue*), and may occur in the plural (*goats, abaci*).
count noun Many languages, including English, divide common nouns into **count nouns** and
mass noun **mass nouns**. Count nouns can occur in the singular and plural (*goat/goats, relationship/relationships*) and can be counted (*one goat, two goats*). Mass nouns are

proper noun used when something is conceptualized as a homogeneous group. So *snow, salt, and communism* are not counted (i.e., **two snows* or **two communisms*). **Proper nouns**, like *Regina, Colorado, and IBM*, are names of specific persons or entities.

verb	Verbs refer to actions and processes, including main verbs like <i>draw</i> , <i>provide</i> , and <i>go</i> . English verbs have inflections (non-third-person-singular (<i>eat</i>), third-person-singular (<i>eats</i>), progressive (<i>eating</i>), past participle (<i>eaten</i>)). While many scholars believe that all human languages have the categories of noun and verb, others have argued that some languages, such as Riau Indonesian and Tongan, don't even make this distinction (Broschart 1997; Evans 2000; Gil 2000).
adjective	Adjectives often describe properties or qualities of nouns, like color (<i>white</i> , <i>black</i>), age (<i>old</i> , <i>young</i>), and value (<i>good</i> , <i>bad</i>), but there are languages without adjectives. In Korean, for example, the words corresponding to English adjectives act as a subclass of verbs, so what is in English an adjective “beautiful” acts in Korean like a verb meaning “to be beautiful”.
adverb	Adverbs are a hodge-podge. All the italicized words in this example are adverbs: <i>Actually</i> , I ran <i>home</i> <i>extremely</i> <i>quickly</i> <i>yesterday</i>
locative degree	Adverbs generally modify something (often verbs, hence the name “adverb”, but also other adverbs and entire verb phrases). Directional adverbs or locative adverbs (<i>home</i> , <i>here</i> , <i>downhill</i>) specify the direction or location of some action; degree adverbs (<i>extremely</i> , <i>very</i> , <i>somewhat</i>) specify the extent of some action, process, or property; manner adverbs (<i>slowly</i> , <i>slinkily</i> , <i>delicately</i>) describe the manner of some action or process; and temporal adverbs describe the time that some action or event took place (<i>yesterday</i> , <i>Monday</i>).
manner temporal	
interjection	Interjections (<i>oh</i> , <i>hey</i> , <i>alas</i> , <i>uh</i> , <i>um</i>) are a smaller open class that also includes greetings (<i>hello</i> , <i>goodbye</i>) and question responses (<i>yes</i> , <i>no</i> , <i>uh-huh</i>).
preposition	English adpositions occur before nouns, hence are called prepositions . They can indicate spatial or temporal relations, whether literal (<i>on it</i> , <i>before then</i> , <i>by the house</i>) or metaphorical (<i>on time</i> , <i>with gusto</i> , <i>beside herself</i>), and relations like marking the agent in <i>Hamlet was written by Shakespeare</i> .
particle	A particle resembles a preposition or an adverb and is used in combination with a verb. Particles often have extended meanings that aren't quite the same as the prepositions they resemble, as in the particle <i>over</i> in <i>she turned the paper over</i> . A verb and a particle acting as a single unit is called a phrasal verb . The meaning of phrasal verbs is often non-compositional —not predictable from the individual meanings of the verb and the particle. Thus, <i>turn down</i> means ‘reject’, <i>rule out</i> ‘eliminate’, and <i>go on</i> ‘continue’.
phrasal verb	
determiner article	Determiners like <i>this</i> and <i>that</i> (<i>this chapter</i> , <i>that page</i>) can mark the start of an English noun phrase. Articles like <i>a</i> , <i>an</i> , and <i>the</i> , are a type of determiner that mark discourse properties of the noun and are quite frequent; <i>the</i> is the most common word in written English, with <i>a</i> and <i>an</i> right behind.
conjunction	Conjunctions join two phrases, clauses, or sentences. Coordinating conjunctions like <i>and</i> , <i>or</i> , and <i>but</i> join two elements of equal status. Subordinating conjunctions are used when one of the elements has some embedded status. For example, the subordinating conjunction <i>that</i> in “ <i>I thought that you might like some milk</i> ” links the main clause <i>I thought</i> with the subordinate clause <i>you might like some milk</i> . This clause is called subordinate because this entire clause is the “content” of the main verb <i>thought</i> . Subordinating conjunctions like <i>that</i> which link a verb to its argument in this way are also called complementizers .
complementizer pronoun	Pronouns act as a shorthand for referring to an entity or event. Personal pronouns refer to persons or entities (<i>you</i> , <i>she</i> , <i>I</i> , <i>it</i> , <i>me</i> , etc.). Possessive pronouns are forms of personal pronouns that indicate either actual possession or more often just an abstract relation between the person and some object (<i>my</i> , <i>your</i> , <i>his</i> , <i>her</i> , <i>its</i> , <i>one's</i> , <i>our</i> , <i>their</i>). Wh-pronouns (<i>what</i> , <i>who</i> , <i>whom</i> , <i>whoever</i>) are used in certain question
wh	

forms, or act as complementizers (*Frida, who married Diego...*).

auxiliary **Auxiliary** verbs mark semantic features of a main verb such as its tense, whether it is completed (aspect), whether it is negated (polarity), and whether an action is necessary, possible, suggested, or desired (mood). English auxiliaries include the **copula** verb *be*, the two verbs *do* and *have*, forms, as well as **modal verbs** used to mark the mood associated with the event depicted by the main verb: *can* indicates ability or possibility, *may* permission or possibility, *must* necessity.

An English-specific tagset, the 45-tag Penn Treebank tagset (Marcus et al., 1993), shown in Fig. 8.2, has been used to label many syntactically annotated corpora like the Penn Treebank corpora, so is worth knowing about.

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coord. conj.	<i>and, but, or</i>	NNP	proper noun, sing.	<i>IBM</i>	TO	“to”	<i>to</i>
CD	cardinal number	<i>one, two</i>	NNPS	proper noun, plu.	<i>Carolinas</i>	UH	interjection	<i>ah, oops</i>
DT	determiner	<i>a, the</i>	NNS	noun, plural	<i>llamas</i>	VB	verb base	<i>eat</i>
EX	existential ‘there’	<i>there</i>	PDT	predeterminer	<i>all, both</i>	VBD	verb past tense	<i>ate</i>
FW	foreign word	<i>mea culpa</i>	POS	possessive ending	<i>’s</i>	VBG	verb gerund	<i>eating</i>
IN	preposition/ subordin-conj	<i>of, in, by</i>	PRP	personal pronoun	<i>I, you, he</i>	VBN	verb past partici- ple	<i>eaten</i>
JJ	adjective	<i>yellow</i>	PRP\$	possess. pronoun	<i>your, one’s</i>	VBP	verb non-3sg-pr	<i>eat</i>
JJR	comparative adj	<i>bigger</i>	RB	adverb	<i>quickly</i>	VBZ	verb 3sg pres	<i>eats</i>
JJS	superlative adj	<i>wildest</i>	RBR	comparative adv	<i>faster</i>	WDT	wh-determ.	<i>which, that</i>
LS	list item marker	<i>1, 2, One</i>	RBS	superlatv. adv	<i>fastest</i>	WP	wh-pronoun	<i>what, who</i>
MD	modal	<i>can, should</i>	RP	particle	<i>up, off</i>	WP\$	wh-possess.	<i>whose</i>
NN	sing or mass noun	<i>llama</i>	SYM	symbol	<i>+, %, &</i>	WRB	wh-adverb	<i>how, where</i>

Figure 8.2 Penn Treebank part-of-speech tags.

Below we show some examples with each word tagged according to both the UD and Penn tagsets. Notice that the Penn tagset distinguishes tense and participles on verbs, and has a special tag for the existential *there* construction in English. Note that since *New England Journal of Medicine* is a proper noun, both tagsets mark its component nouns as NNP, including *journal* and *medicine*, which might otherwise be labeled as common nouns (NOUN/NN).

- (8.1) There/**PRO**/**EX** are/**VERB**/**VBP** 70/**NUM**/**CD** children/**NOUN**/**NNS**
there/**ADV**/**RB** ./**PUNC**/.
- (8.2) Preliminary/**ADJ**/**JJ** findings/**NOUN**/**NNS** were/**AUX**/**VBD** reported/**VERB**/**VBN**
in/**ADP**/**IN** today/**NOUN**/**NN** ’s/**PART**/**POS** New/**PROPN**/**NNP**
England/**PROPN**/**NNP** Journal/**PROPN**/**NNP** of/**ADP**/**IN** Medicine/**PROPN**/**NNP**

8.2 Part-of-Speech Tagging

**part-of-speech
tagging**

Part-of-speech tagging is the process of assigning a part-of-speech to each word in a text. The input is a sequence $x_1, x_2, \dots, x_n$ of (tokenized) words and a tagset, and the output is a sequence $y_1, y_2, \dots, y_n$ of tags, each output y_i corresponding exactly to one input x_i , as shown in the intuition in Fig. 8.3.

ambiguous

Tagging is a **disambiguation** task; words are **ambiguous**—have more than one possible part-of-speech—and the goal is to find the correct tag for the situation. For example, *book* can be a verb (*book that flight*) or a noun (*hand me that book*). *That* can be a determiner (*Does that flight serve dinner*) or a complementizer (*I*

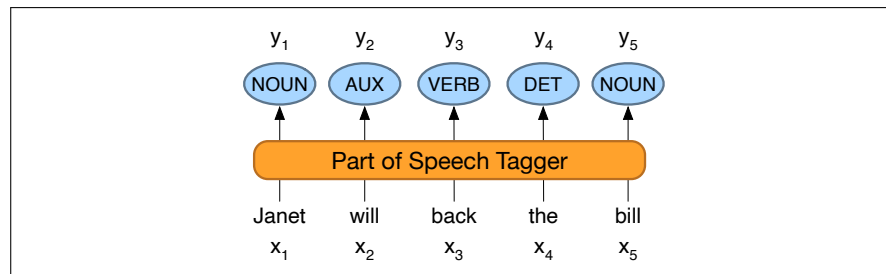


Figure 8.3 The task of part-of-speech tagging: mapping from input words $x_1, x_2, \dots, x_n$ to output POS tags $y_1, y_2, \dots, y_n$.

ambiguity
resolution

thought that your flight was earlier). The goal of POS-tagging is to **resolve** these ambiguities, choosing the proper tag for the context.

accuracy

The **accuracy** of part-of-speech tagging algorithms (the percentage of test set tags that match human gold labels) is extremely high. One study found accuracies over 97% across 15 languages from the Universal Dependency (UD) treebank (Wu and Dredze, 2019). Accuracies on various English treebanks are also 97% (no matter the algorithm; HMMs, CRFs, BERT perform similarly). This 97% number is also about the human performance on this task, at least for English (Manning, 2011).

Types:	WSJ	Brown
Unambiguous (1 tag)	44,432 (86%)	45,799 (85%)
Ambiguous (2+ tags)	7,025 (14%)	8,050 (15%)
Tokens:		
Unambiguous (1 tag)	577,421 (45%)	384,349 (33%)
Ambiguous (2+ tags)	711,780 (55%)	786,646 (67%)

Figure 8.4 Tag ambiguity in the Brown and WSJ corpora (Treebank-3 45-tag tagset).

We'll introduce algorithms for the task in the next few sections, but first let's explore the task. Exactly how hard is it? Fig. 8.4 shows that most word types (85-86%) are unambiguous (*Janet* is always NNP, *hesitantly* is always RB). But the ambiguous words, though accounting for only 14-15% of the vocabulary, are very common, and 55-67% of word tokens in running text are ambiguous. Particularly ambiguous common words include *that*, *back*, *down*, *put* and *set*; here are some examples of the 6 different parts of speech for the word *back*:

earnings growth took a **back/JJ** seat
 a small building in the **back/NN**
 a clear majority of senators **back/VBP** the bill
 Dave began to **back/VB** toward the door
 enable the country to buy **back/RP** debt
 I was twenty-one **back/RB** then

Nonetheless, many words are easy to disambiguate, because their different tags aren't equally likely. For example, *a* can be a determiner or the letter *a*, but the determiner sense is much more likely.

This idea suggests a useful **baseline**: given an ambiguous word, choose the tag which is **most frequent** in the training corpus. This is a key concept:

Most Frequent Class Baseline: Always compare a classifier against a baseline at least as good as the most frequent class baseline (assigning each token to the class it occurred in most often in the training set).

The most-frequent-tag baseline has an accuracy of about 92%¹. The baseline thus differs from the state-of-the-art and human ceiling (97%) by only 5%.

8.3 Named Entities and Named Entity Tagging

Part of speech tagging can tell us that words like *Janet*, *Stanford University*, and *Colorado* are all proper nouns; being a proper noun is a grammatical property of these words. But viewed from a semantic perspective, these proper nouns refer to different kinds of entities: Janet is a person, Stanford University is an organization, and Colorado is a location.

named entity

named entity
recognition
NER

A **named entity** is, roughly speaking, anything that can be referred to with a proper name: a person, a location, an organization. The task of **named entity recognition (NER)** is to find spans of text that constitute proper names and tag the type of the entity. Four entity tags are most common: **PER** (person), **LOC** (location), **ORG** (organization), or **GPE** (geo-political entity). However, the term **named entity** is commonly extended to include things that aren't entities per se, including dates, times, and other kinds of temporal expressions, and even numerical expressions like prices. Here's an example of the output of an NER tagger:

Citing high fuel prices, [ORG United Airlines] said [TIME Friday] it has increased fares by [MONEY \$6] per round trip on flights to some cities also served by lower-cost carriers. [ORG American Airlines], a unit of [ORG AMR Corp.], immediately matched the move, spokesman [PER Tim Wagner] said. [ORG United], a unit of [ORG UAL Corp.], said the increase took effect [TIME Thursday] and applies to most routes where it competes against discount carriers, such as [LOC Chicago] to [LOC Dallas] and [LOC Denver] to [LOC San Francisco].

The text contains 13 mentions of named entities including 5 organizations, 4 locations, 2 times, 1 person, and 1 mention of money. Figure 8.5 shows typical generic named entity types. Many applications will also need to use specific entity types like proteins, genes, commercial products, or works of art.

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	Turing is a giant of computer science.
Organization	ORG	companies, sports teams	The IPCC warned about the cyclone.
Location	LOC	regions, mountains, seas	Mt. Sanitas is in Sunshine Canyon .
Geo-Political Entity	GPE	countries, states	Palo Alto is raising the fees for parking.

Figure 8.5 A list of generic named entity types with the kinds of entities they refer to.

Named entity tagging is a useful first step in lots of natural language processing tasks. In sentiment analysis we might want to know a consumer's sentiment toward a particular entity. Entities are a useful first stage in question answering, or for linking text to information in structured knowledge sources like Wikipedia. And named entity tagging is also central to tasks involving building semantic representations, like extracting events and the relationship between participants.

Unlike part-of-speech tagging, where there is no segmentation problem since each word gets one tag, the task of named entity recognition is to find and label *spans* of text, and is difficult partly because of the ambiguity of segmentation; we

¹ In English, on the WSJ corpus, tested on sections 22-24.

need to decide what's an entity and what isn't, and where the boundaries are. Indeed, most words in a text will not be named entities. Another difficulty is caused by type ambiguity. The mention JFK can refer to a person, the airport in New York, or any number of schools, bridges, and streets around the United States. Some examples of this kind of cross-type confusion are given in Figure 8.6.

[PER Washington] was born into slavery on the farm of James Burroughs.
 [ORG Washington] went up 2 games to 1 in the four-game series.
 Blair arrived in [LOC Washington] for what may well be his last state visit.
 In June, [GPE Washington] passed a primary seatbelt law.

Figure 8.6 Examples of type ambiguities in the use of the name *Washington*.

The standard approach to sequence labeling for a span-recognition problem like NER is **BIO** tagging (Ramshaw and Marcus, 1995). This is a method that allows us to treat NER like a word-by-word sequence labeling task, via tags that capture both the boundary and the named entity type. Consider the following sentence:

[PER Jane Villanueva] of [ORG United], a unit of [ORG United Airlines Holding], said the fare applies to the [LOC Chicago] route.

BIO Figure 8.7 shows the same excerpt represented with **BIO** tagging, as well as variants called **IO** tagging and **BIOES** tagging. In **BIO** tagging we label any token that *begins* a span of interest with the label B, tokens that occur *inside* a span are tagged with an I, and any tokens outside of any span of interest are labeled O. While there is only one O tag, we'll have distinct B and I tags for each named entity class. The number of tags is thus $2n + 1$ tags, where n is the number of entity types. **BIO** tagging can represent exactly the same information as the bracketed notation, but has the advantage that we can represent the task in the same simple sequence modeling way as part-of-speech tagging: assigning a single label y_i to each input word x_i :

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Figure 8.7 NER as a sequence model, showing IO, BIO, and BIOES taggings.

We've also shown two variant tagging schemes: **IO** tagging, which loses some information by eliminating the B tag, and **BIOES** tagging, which adds an end tag E for the end of a span, and a span tag S for a span consisting of only one word. A sequence labeler (HMM, CRF, RNN, Transformer, etc.) is trained to label each token in a text with tags that indicate the presence (or absence) of particular kinds of named entities.

8.4 HMM Part-of-Speech Tagging

In this section we introduce our first sequence labeling algorithm, the Hidden Markov Model, and show how to apply it to part-of-speech tagging. Recall that a sequence labeler is a model whose job is to assign a label to each unit in a sequence, thus mapping a sequence of observations to a sequence of labels of the same length. The HMM is a classic model that introduces many of the key concepts of sequence modeling that we will see again in more modern models.

An HMM is a probabilistic sequence model: given a sequence of units (words, letters, morphemes, sentences, whatever), it computes a probability distribution over possible sequences of labels and chooses the best label sequence.

8.4.1 Markov Chains

Markov chain

The HMM is based on augmenting the Markov chain. A **Markov chain** is a model that tells us something about the probabilities of sequences of random variables, *states*, each of which can take on values from some set. These sets can be words, or tags, or symbols representing anything, for example the weather. A Markov chain makes a very strong assumption that if we want to predict the future in the sequence, all that matters is the current state. All the states before the current state have no impact on the future except via the current state. It's as if to predict tomorrow's weather you could examine today's weather but you weren't allowed to look at yesterday's weather.

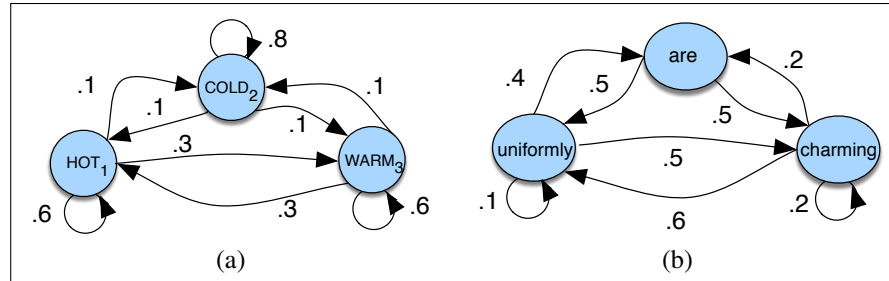


Figure 8.8 A Markov chain for weather (a) and one for words (b), showing states and transitions. A start distribution π is required; setting $\pi = [0.1, 0.7, 0.2]$ for (a) would mean a probability 0.7 of starting in state 2 (cold), probability 0.1 of starting in state 1 (hot), etc.

Markov assumption

More formally, consider a sequence of state variables $q_1, q_2, \dots, q_i$. A Markov model embodies the **Markov assumption** on the probabilities of this sequence: that when predicting the future, the past doesn't matter, only the present.

$$\textbf{Markov Assumption: } P(q_i = a | q_1 \dots q_{i-1}) = P(q_i = a | q_{i-1}) \quad (8.3)$$

Figure 8.8a shows a Markov chain for assigning a probability to a sequence of weather events, for which the vocabulary consists of HOT, COLD, and WARM. The states are represented as nodes in the graph, and the transitions, with their probabilities, as edges. The transitions are probabilities: the values of arcs leaving a given state must sum to 1. Figure 8.8b shows a Markov chain for assigning a probability to a sequence of words $w_1 \dots w_i$. This Markov chain should be familiar; in fact, it represents a bigram language model, with each edge expressing the probability $p(w_i | w_j)$! Given the two models in Fig. 8.8, we can assign a probability to any sequence from our vocabulary.

Formally, a Markov chain is specified by the following components:

$Q = q_1 q_2 \dots q_N$	a set of N states
$A = a_{11} a_{12} \dots a_{N1} \dots a_{NN}$	a transition probability matrix A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^n a_{ij} = 1 \quad \forall i$
$\pi = \pi_1, \pi_2, \dots, \pi_N$	an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

Before you go on, use the sample probabilities in Fig. 8.8a (with $\pi = [0.1, 0.7, 0.2]$) to compute the probability of each of the following sequences:

(8.4) hot hot hot hot

(8.5) cold hot cold hot

What does the difference in these probabilities tell you about a real-world weather fact encoded in Fig. 8.8a?

8.4.2 The Hidden Markov Model

A Markov chain is useful when we need to compute a probability for a sequence of observable events. In many cases, however, the events we are interested in are **hidden**: we don't observe them directly. For example we don't normally observe part-of-speech tags in a text. Rather, we see words, and must infer the tags from the word sequence. We call the tags **hidden** because they are not observed.

hidden
hidden Markov
model

A **hidden Markov model (HMM)** allows us to talk about both *observed* events (like words that we see in the input) and *hidden* events (like part-of-speech tags) that we think of as causal factors in our probabilistic model. An HMM is specified by the following components:

$Q = q_1 q_2 \dots q_N$	a set of N states
$A = a_{11} \dots a_{ij} \dots a_{NN}$	a transition probability matrix A , each a_{ij} representing the probability of moving from state i to state j , s.t. $\sum_{j=1}^N a_{ij} = 1 \quad \forall i$
$O = o_1 o_2 \dots o_T$	a sequence of T observations , each one drawn from a vocabulary $V = v_1, v_2, \dots, v_V$
$B = b_i(o_i)$	a sequence of observation likelihoods , also called emission probabilities , each expressing the probability of an observation o_i being generated from a state q_i
$\pi = \pi_1, \pi_2, \dots, \pi_N$	an initial probability distribution over states. π_i is the probability that the Markov chain will start in state i . Some states j may have $\pi_j = 0$, meaning that they cannot be initial states. Also, $\sum_{i=1}^n \pi_i = 1$

A first-order hidden Markov model instantiates two simplifying assumptions. First, as with a first-order Markov chain, the probability of a particular state depends only on the previous state:

$$\textbf{Markov Assumption: } P(q_i | q_1, \dots, q_{i-1}) = P(q_i | q_{i-1}) \quad (8.6)$$

Second, the probability of an output observation o_i depends only on the state that produced the observation q_i and not on any other states or any other observations:

$$\textbf{Output Independence: } P(o_i | q_1, \dots, q_i, \dots, q_T, o_1, \dots, o_i, \dots, o_T) = P(o_i | q_i) \quad (8.7)$$

8.4.3 The components of an HMM tagger

Let's start by looking at the pieces of an HMM tagger, and then we'll see how to use it to tag. An HMM has two components, the A and B probabilities.

The A matrix contains the tag transition probabilities $P(t_i|t_{i-1})$ which represent the probability of a tag occurring given the previous tag. For example, modal verbs like *will* are very likely to be followed by a verb in the base form, a VB, like *race*, so we expect this probability to be high. We compute the maximum likelihood estimate of this transition probability by counting, out of the times we see the first tag in a labeled corpus, how often the first tag is followed by the second:

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})} \quad (8.8)$$

In the WSJ corpus, for example, MD occurs 13124 times of which it is followed by VB 10471, for an MLE estimate of

$$P(VB|MD) = \frac{C(MD, VB)}{C(MD)} = \frac{10471}{13124} = .80 \quad (8.9)$$

Let's walk through an example, seeing how these probabilities are estimated and used in a sample tagging task, before we return to the algorithm for decoding.

In HMM tagging, the probabilities are estimated by counting on a tagged training corpus. For this example we'll use the tagged WSJ corpus.

The B emission probabilities, $P(w_i|t_i)$, represent the probability, given a tag (say MD), that it will be associated with a given word (say *will*). The MLE of the emission probability is

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)} \quad (8.10)$$

Of the 13124 occurrences of MD in the WSJ corpus, it is associated with *will* 4046 times:

$$P(will|MD) = \frac{C(MD, will)}{C(MD)} = \frac{4046}{13124} = .31 \quad (8.11)$$

We saw this kind of Bayesian modeling in Chapter 4; recall that this likelihood term is not asking “which is the most likely tag for the word *will*?” That would be the posterior $P(MD|will)$. Instead, $P(will|MD)$ answers the slightly counterintuitive question “If we were going to generate a MD, how likely is it that this modal would be *will*?”

The A transition probabilities, and B observation likelihoods of the HMM are illustrated in Fig. 8.9 for three states in an HMM part-of-speech tagger; the full tagger would have one state for each tag.

8.4.4 HMM tagging as decoding

For any model, such as an HMM, that contains hidden variables, the task of determining the hidden variables sequence corresponding to the sequence of observations is called **decoding**. More formally,

Decoding: Given as input an HMM $\lambda = (A, B)$ and a sequence of observations $O = o_1, o_2, \dots, o_T$, find the most probable sequence of states $Q = q_1 q_2 q_3 \dots q_T$.

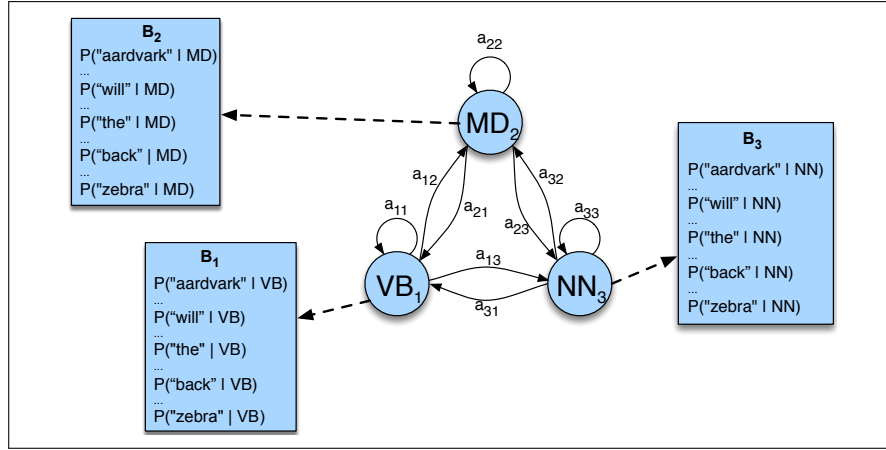


Figure 8.9 An illustration of the two parts of an HMM representation: the A transition probabilities used to compute the prior probability, and the B observation likelihoods that are associated with each state, one likelihood for each possible observation word.

For part-of-speech tagging, the goal of HMM decoding is to choose the tag sequence $t_1 \dots t_n$ that is most probable given the observation sequence of n words $w_1 \dots w_n$:

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} P(t_1 \dots t_n | w_1 \dots w_n) \quad (8.12)$$

The way we'll do this in the HMM is to use Bayes' rule to instead compute:

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} \frac{P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n)}{P(w_1 \dots w_n)} \quad (8.13)$$

Furthermore, we simplify Eq. 8.13 by dropping the denominator $P(w_1^n)$:

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n) \quad (8.14)$$

HMM taggers make two further simplifying assumptions. The first is that the probability of a word appearing depends only on its own tag and is independent of neighboring words and tags:

$$P(w_1 \dots w_n | t_1 \dots t_n) \approx \prod_{i=1}^n P(w_i | t_i) \quad (8.15)$$

The second assumption, the **bigram** assumption, is that the probability of a tag is dependent only on the previous tag, rather than the entire tag sequence;

$$P(t_1 \dots t_n) \approx \prod_{i=1}^n P(t_i | t_{i-1}) \quad (8.16)$$

Plugging the simplifying assumptions from Eq. 8.15 and Eq. 8.16 into Eq. 8.14 results in the following equation for the most probable tag sequence from a bigram tagger:

$$\hat{t}_{1:n} = \operatorname{argmax}_{t_1 \dots t_n} P(t_1 \dots t_n | w_1 \dots w_n) \approx \operatorname{argmax}_{t_1 \dots t_n} \prod_{i=1}^n \overbrace{P(w_i | t_i)}^{\text{emission}} \overbrace{P(t_i | t_{i-1})}^{\text{transition}} \quad (8.17)$$

The two parts of Eq. 8.17 correspond neatly to the B **emission probability** and A **transition probability** that we just defined above!

8.4.5 The Viterbi Algorithm

**Viterbi
algorithm**

The decoding algorithm for HMMs is the **Viterbi algorithm** shown in Fig. 8.10. As an instance of **dynamic programming**, Viterbi resembles the dynamic programming **minimum edit distance** algorithm of Chapter 2.

```

function VITERBI(observations of len  $T$ , state-graph of len  $N$ ) returns best-path, path-prob

create a path probability matrix viterbi[ $N, T$ ]
for each state  $s$  from 1 to  $N$  do                                ; initialization step
    viterbi[ $s, 1$ ]  $\leftarrow \pi_s * b_s(o_1)$ 
    backpointer[ $s, 1$ ]  $\leftarrow 0$ 
for each time step  $t$  from 2 to  $T$  do                            ; recursion step
    for each state  $s$  from 1 to  $N$  do
        viterbi[ $s, t$ ]  $\leftarrow \max_{s'=1}^N \textit{viterbi}[s', t-1] * a_{s',s} * b_s(o_t)$ 
        backpointer[ $s, t$ ]  $\leftarrow \operatorname{argmax}_{s'=1}^N \textit{viterbi}[s', t-1] * a_{s',s} * b_s(o_t)$ 
    bestpathprob  $\leftarrow \max_{s=1}^N \textit{viterbi}[s, T]$                         ; termination step
    bestpathpointer  $\leftarrow \operatorname{argmax}_{s=1}^N \textit{viterbi}[s, T]$                 ; termination step
    bestpath  $\leftarrow$  the path starting at state bestpathpointer, that follows backpointer[] to states back in time
    return bestpath, bestpathprob

```

Figure 8.10 Viterbi algorithm for finding the optimal sequence of tags. Given an observation sequence and an HMM $\lambda = (A, B)$, the algorithm returns the state path through the HMM that assigns maximum likelihood to the observation sequence.

The Viterbi algorithm first sets up a probability matrix or **lattice**, with one column for each observation o_t and one row for each state in the state graph. Each column thus has a cell for each state q_i in the single combined automaton. Figure 8.11 shows an intuition of this lattice for the sentence *Janet will back the bill*.

Each cell of the lattice, $v_t(j)$, represents the probability that the HMM is in state j after seeing the first t observations and passing through the most probable state sequence $q_1, \dots, q_{t-1}$, given the HMM λ . The value of each cell $v_t(j)$ is computed by recursively taking the most probable path that could lead us to this cell. Formally, each cell expresses the probability

$$v_t(j) = \max_{q_1, \dots, q_{t-1}} P(q_1 \dots q_{t-1}, o_1, o_2 \dots o_t, q_t = j | \lambda) \quad (8.18)$$

We represent the most probable path by taking the maximum over all possible previous state sequences $\max_{q_1, \dots, q_{t-1}}$. Like other dynamic programming algorithms, Viterbi fills each cell recursively. Given that we had already computed the probability of being in every state at time $t-1$, we compute the Viterbi probability by taking the most probable of the extensions of the paths that lead to the current cell. For a given state q_j at time t , the value $v_t(j)$ is computed as

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t) \quad (8.19)$$

The three factors that are multiplied in Eq. 8.19 for extending the previous paths to compute the Viterbi probability at time t are

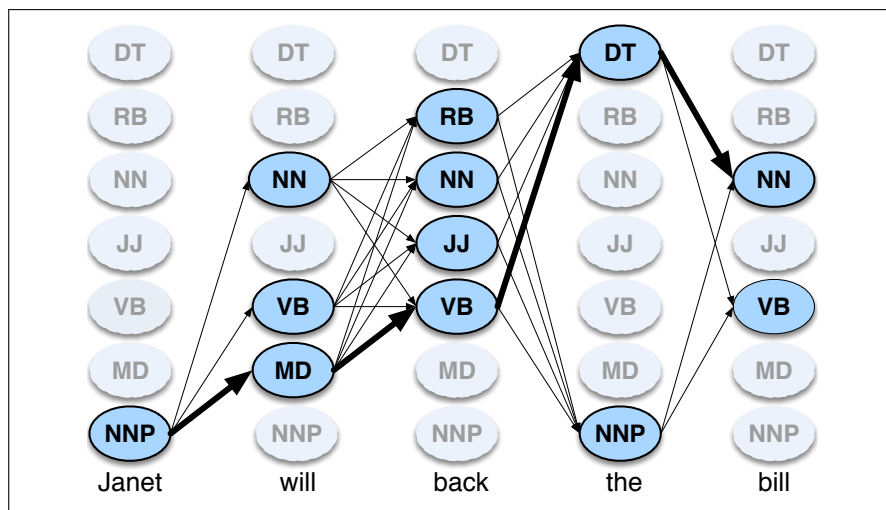


Figure 8.11 A sketch of the lattice for *Janet will back the bill*, showing the possible tags (q_i) for each word and highlighting the path corresponding to the correct tag sequence through the hidden states. States (parts of speech) which have a zero probability of generating a particular word according to the B matrix (such as the probability that a determiner DT will be realized as *Janet*) are greyed out.

$v_{t-1}(i)$	the previous Viterbi path probability from the previous time step
a_{ij}	the transition probability from previous state q_i to current state q_j
$b_j(o_t)$	the state observation likelihood of the observation symbol o_t given the current state j

8.4.6 Working through an example

Let's tag the sentence *Janet will back the bill*; the goal is the correct series of tags (see also Fig. 8.11):

(8.20) Janet/NNP will/MD back/VB the/DT bill/NN

	NNP	MD	VB	JJ	NN	RB	DT
$\langle s \rangle$	0.2767	0.0006	0.0031	0.0453	0.0449	0.0510	0.2026
NNP	0.3777	0.0110	0.0009	0.0084	0.0584	0.0090	0.0025
MD	0.0008	0.0002	0.7968	0.0005	0.0008	0.1698	0.0041
VB	0.0322	0.0005	0.0050	0.0837	0.0615	0.0514	0.2231
JJ	0.0366	0.0004	0.0001	0.0733	0.4509	0.0036	0.0036
NN	0.0096	0.0176	0.0014	0.0086	0.1216	0.0177	0.0068
RB	0.0068	0.0102	0.1011	0.1012	0.0120	0.0728	0.0479
DT	0.1147	0.0021	0.0002	0.2157	0.4744	0.0102	0.0017

Figure 8.12 The A transition probabilities $P(t_i|t_{i-1})$ computed from the WSJ corpus without smoothing. Rows are labeled with the conditioning event; thus $P(VB|MD)$ is 0.7968.

Let the HMM be defined by the two tables in Fig. 8.12 and Fig. 8.13. Figure 8.12 lists the a_{ij} probabilities for transitioning between the hidden states (part-of-speech tags). Figure 8.13 expresses the $b_i(o_t)$ probabilities, the *observation* likelihoods of words given tags. This table is (slightly simplified) from counts in the WSJ corpus. So the word *Janet* only appears as an NNP, *back* has 4 possible parts of speech, and

	Janet	will	back	the	bill
NNP	0.000032	0	0	0.000048	0
MD	0	0.308431	0	0	0
VB	0	0.000028	0.000672	0	0.000028
JJ	0	0	0.000340	0	0
NN	0	0.000200	0.000223	0	0.002337
RB	0	0	0.010446	0	0
DT	0	0	0	0.506099	0

Figure 8.13 Observation likelihoods B computed from the WSJ corpus without smoothing, simplified slightly.

the word *the* can appear as a determiner or as an NNP (in titles like “Somewhere Over the Rainbow” all words are tagged as NNP).

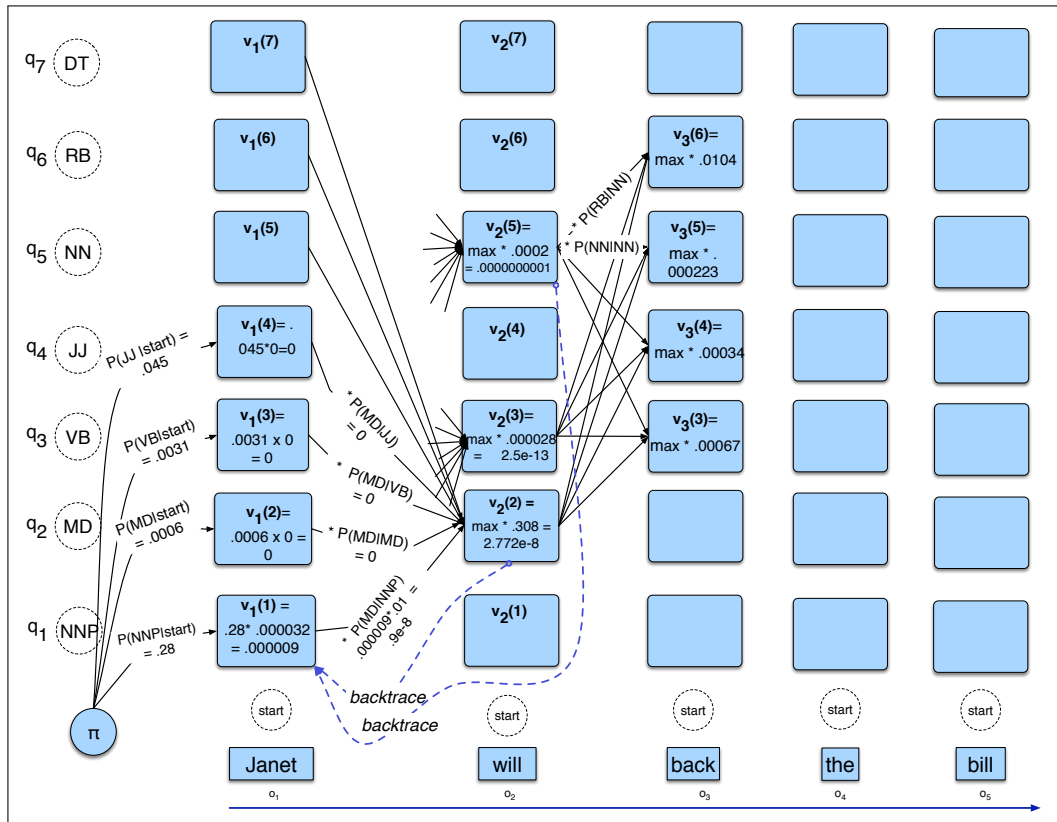


Figure 8.14 The first few entries in the individual state columns for the Viterbi algorithm. Each cell keeps the probability of the best path so far and a pointer to the previous cell along that path. We have only filled out columns 1 and 2; to avoid clutter most cells with value 0 are left empty. The rest is left as an exercise for the reader. After the cells are filled in, backtracing from the *end* state, we should be able to reconstruct the correct state sequence NNP MD VB DT NN.

Figure 8.14 shows a fleshed-out version of the sketch we saw in Fig. 8.11, the Viterbi lattice for computing the best hidden state sequence for the observation sequence *Janet will back the bill*.

There are $N = 5$ state columns. We begin in column 1 (for the word *Janet*) by setting the Viterbi value in each cell to the product of the π transition probability (the start probability for that state i , which we get from the $\langle s \rangle$ entry of Fig. 8.12),

and the observation likelihood of the word *Janet* given the tag for that cell. Most of the cells in the column are zero since the word *Janet* cannot be any of those tags. The reader should find this in Fig. 8.14.

Next, each cell in the *will* column gets updated. For each state, we compute the value $viterbi[s, t]$ by taking the maximum over the extensions of all the paths from the previous column that lead to the current cell according to Eq. 8.19. We have shown the values for the MD, VB, and NN cells. Each cell gets the max of the 7 values from the previous column, multiplied by the appropriate transition probability; as it happens in this case, most of them are zero from the previous column. The remaining value is multiplied by the relevant observation probability, and the (trivial) max is taken. In this case the final value, $2.772e-8$, comes from the NNP state at the previous column. The reader should fill in the rest of the lattice in Fig. 8.14 and backtrack to see whether or not the Viterbi algorithm returns the gold state sequence NNP MD VB DT NN.

8.5 Conditional Random Fields (CRFs)

unknown words

While the HMM is a useful and powerful model, it turns out that HMMs need a number of augmentations to achieve high accuracy. For example, in POS tagging as in other tasks, we often run into **unknown words**: proper names and acronyms are created very often, and even new common nouns and verbs enter the language at a surprising rate. It would be great to have ways to add arbitrary features to help with this, perhaps based on capitalization or morphology (words starting with capital letters are likely to be proper nouns, words ending with *-ed* tend to be past tense (VBD or VBN), etc.) Or knowing the previous or following words might be a useful feature (if the previous word is *the*, the current tag is unlikely to be a verb).

Although we could try to hack the HMM to find ways to incorporate some of these, in general it's hard for generative models like HMMs to add arbitrary features directly into the model in a clean way. We've already seen a model for combining arbitrary features in a principled way: log-linear models like the logistic regression model of Chapter 5! But logistic regression isn't a sequence model; it assigns a class to a single observation.

CRF

Luckily, there is a discriminative sequence model based on log-linear models: the **conditional random field (CRF)**. We'll describe here the **linear chain CRF**, the version of the CRF most commonly used for language processing, and the one whose conditioning closely matches the HMM.

Assuming we have a sequence of input words $X = x_1 \dots x_n$ and want to compute a sequence of output tags $Y = y_1 \dots y_n$. In an HMM to compute the best tag sequence that maximizes $P(Y|X)$ we rely on Bayes' rule and the likelihood $P(X|Y)$:

$$\begin{aligned} \hat{Y} &= \operatorname{argmax}_Y p(Y|X) \\ &= \operatorname{argmax}_Y p(X|Y)p(Y) \\ &= \operatorname{argmax}_Y \prod_i p(x_i|y_i) \prod_i p(y_i|y_{i-1}) \end{aligned} \quad (8.21)$$

In a CRF, by contrast, we compute the posterior $p(Y|X)$ directly, training the CRF

to discriminate among the possible tag sequences:

$$\hat{Y} = \operatorname{argmax}_{Y \in \mathcal{Y}} P(Y|X) \quad (8.22)$$

However, the CRF does not compute a probability for each tag at each time step. Instead, at each time step the CRF computes log-linear functions over a set of relevant features, and these local features are aggregated and normalized to produce a global probability for the whole sequence.

Let's introduce the CRF more formally, again using X and Y as the input and output sequences. A CRF is a log-linear model that assigns a probability to an entire output (tag) sequence Y , out of all possible sequences $\mathcal{Y}$, given the entire input (word) sequence X . We can think of a CRF as like a giant version of what multinomial logistic regression does for a single token. Recall that the feature function f in regular multinomial logistic regression can be viewed as a function of a tuple: a token x and a label y (page 89). In a CRF, the function F maps an entire input sequence X and an entire output sequence Y to a feature vector. Let's assume we have K features, with a weight w_k for each feature F_k :

$$p(Y|X) = \frac{\exp\left(\sum_{k=1}^K w_k F_k(X, Y)\right)}{\sum_{Y' \in \mathcal{Y}} \exp\left(\sum_{k=1}^K w_k F_k(X, Y')\right)} \quad (8.23)$$

It's common to also describe the same equation by pulling out the denominator into a function $Z(X)$:

$$p(Y|X) = \frac{1}{Z(X)} \exp\left(\sum_{k=1}^K w_k F_k(X, Y)\right) \quad (8.24)$$

$$Z(X) = \sum_{Y' \in \mathcal{Y}} \exp\left(\sum_{k=1}^K w_k F_k(X, Y')\right) \quad (8.25)$$

We'll call these K functions $F_k(X, Y)$ **global features**, since each one is a property of the entire input sequence X and output sequence Y . We compute them by decomposing into a sum of **local** features for each position i in Y :

$$F_k(X, Y) = \sum_{i=1}^n f_k(y_{i-1}, y_i, X, i) \quad (8.26)$$

linear chain
CRF

Each of these local features f_k in a linear-chain CRF is allowed to make use of the current output token y_i , the previous output token y_{i-1} , the entire input string X (or any subpart of it), and the current position i . This constraint to only depend on the current and previous output tokens y_i and y_{i-1} are what characterizes a **linear chain CRF**. As we will see, this limitation makes it possible to use versions of the efficient Viterbi and Forward-Backwards algorithms from the HMM. A general CRF, by contrast, allows a feature to make use of any output token, and are thus necessary for tasks in which the decision depend on distant output tokens, like y_{i-4} . General CRFs require more complex inference, and are less commonly used for language processing.

8.5.1 Features in a CRF POS Tagger

Let's look at some of these features in detail, since the reason to use a discriminative sequence model is that it's easier to incorporate a lot of features.²

Again, in a linear-chain CRF, each local feature f_k at position i can depend on any information from: (y_{i-1}, y_i, X, i) . So some legal features representing common situations might be the following:

$$\begin{aligned} &\mathbb{1}\{x_i = \textit{the}, y_i = \text{DET}\} \\ &\mathbb{1}\{y_i = \text{PROP}, x_{i+1} = \textit{Street}, y_{i-1} = \text{NUM}\} \\ &\mathbb{1}\{y_i = \text{VERB}, y_{i-1} = \text{AUX}\} \end{aligned}$$

For simplicity, we'll assume all CRF features take on the value 1 or 0. Above, we explicitly use the notation $\mathbb{1}\{x\}$ to mean "1 if x is true, and 0 otherwise". From now on, we'll leave off the $\mathbb{1}$ when we define features, but you can assume each feature has it there implicitly.

feature
templates

Although the idea of what features to use is done by the system designer by hand, the specific features are automatically populated by using **feature templates** as we briefly mentioned in Chapter 5. Here are some templates that only use information from (y_{i-1}, y_i, X, i) :

$$\langle y_i, x_i \rangle, \langle y_i, y_{i-1} \rangle, \langle y_i, x_{i-1}, x_{i+2} \rangle$$

These templates automatically populate the set of features from every instance in the training and test set. Thus for our example *Janet/NNP will/MD back/VB the/DT bill/NN*, when x_i is the word *back*, the following features would be generated and have the value 1 (we've assigned them arbitrary feature numbers):

$$\begin{aligned} f_{3743}: y_i = \text{VB} \text{ and } x_i = \textit{back} \\ f_{156}: y_i = \text{VB} \text{ and } y_{i-1} = \text{MD} \\ f_{99732}: y_i = \text{VB} \text{ and } x_{i-1} = \textit{will} \text{ and } x_{i+2} = \textit{bill} \end{aligned}$$

word shape

It's also important to have features that help with unknown words. One of the most important is **word shape** features, which represent the abstract letter pattern of the word by mapping lower-case letters to 'x', upper-case to 'X', numbers to 'd', and retaining punctuation. Thus for example I.M.F. would map to X.X.X. and DC10-30 would map to XXdd-dd. A second class of shorter word shape features is also used. In these features consecutive character types are removed, so words in all caps map to X, words with initial-caps map to Xx, DC10-30 would be mapped to Xd-d but I.M.F would still map to X.X.X. Prefix and suffix features are also useful. In summary, here are some sample feature templates that help with unknown words:

$$\begin{aligned} &x_i \text{ contains a particular prefix (perhaps from all prefixes of length } \leq 2) \\ &x_i \text{ contains a particular suffix (perhaps from all suffixes of length } \leq 2) \\ &x_i \text{'s word shape} \\ &x_i \text{'s short word shape} \end{aligned}$$

For example the word *well-dressed* might generate the following non-zero valued feature values:

² Because in HMMs all computation is based on the two probabilities $P(\text{tag}|\text{tag})$ and $P(\text{word}|\text{tag})$, if we want to include some source of knowledge into the tagging process, we must find a way to encode the knowledge into one of these two probabilities. Each time we add a feature we have to do a lot of complicated conditioning which gets harder and harder as we have more and more such features.

```

prefix( $x_i$ ) = w
prefix( $x_i$ ) = we
suffix( $x_i$ ) = ed
suffix( $x_i$ ) = d
word-shape( $x_i$ ) = xxxx-xxxxxxx
short-word-shape( $x_i$ ) = x-x

```

The known-word templates are computed for every word seen in the training set; the unknown word features can also be computed for all words in training, or only on training words whose frequency is below some threshold. The result of the known-word templates and word-signature features is a very large set of features. Generally a feature cutoff is used in which features are thrown out if they have count < 5 in the training set.

Remember that in a CRF we don't learn weights for each of these local features f_k . Instead, we first sum the values of each local feature (for example feature f_{3743}) over the entire sentence, to create each global feature (for example F_{3743}). It is those global features that will then be multiplied by weight w_{3743} . Thus for training and inference there is always a fixed set of K features with K weights, even though the length of each sentence is different.

8.5.2 Features for CRF Named Entity Recognizers

A CRF for NER makes use of very similar features to a POS tagger, as shown in Figure 8.15.

identity of w_i , identity of neighboring words
 embeddings for w_i , embeddings for neighboring words
 part of speech of w_i , part of speech of neighboring words
 presence of w_i in a **gazetteer**
 w_i contains a particular prefix (from all prefixes of length ≤ 4)
 w_i contains a particular suffix (from all suffixes of length ≤ 4)
 word shape of w_i , word shape of neighboring words
 short word shape of w_i , short word shape of neighboring words
 gazetteer features

Figure 8.15 Typical features for a feature-based NER system.

gazetteer

One feature that is especially useful for locations is a **gazetteer**, a list of place names, often providing millions of entries for locations with detailed geographical and political information.³ This can be implemented as a binary feature indicating a phrase appears in the list. Other related resources like **name-lists**, for example from the United States Census Bureau⁴, can be used, as can other entity dictionaries like lists of corporations or products, although they may not be as helpful as a gazetteer (Mikheev et al., 1999).

The sample named entity token *L'Occitane* would generate the following non-zero valued feature values (assuming that *L'Occitane* is neither in the gazetteer nor the census).

³ www.geonames.org

⁴ www.census.gov

$\text{prefix}(x_i) = \text{L}$ $\text{suffix}(x_i) = \text{tane}$
 $\text{prefix}(x_i) = \text{L}'$ $\text{suffix}(x_i) = \text{ane}$
 $\text{prefix}(x_i) = \text{L}'\text{O}$ $\text{suffix}(x_i) = \text{ne}$
 $\text{prefix}(x_i) = \text{L}'\text{Oc}$ $\text{suffix}(x_i) = \text{e}$
 $\text{word-shape}(x_i) = \text{X}'\text{Xxxxxxxx}$ $\text{short-word-shape}(x_i) = \text{X}'\text{Xx}$

Figure 8.16 illustrates the result of adding part-of-speech tags and some shape information to our earlier example.

Words	POS	Short shape	Gazetteer	BIO Label
Jane	NNP	Xx	0	B-PER
Villanueva	NNP	Xx	1	I-PER
of	IN	x	0	O
United	NNP	Xx	0	B-ORG
Airlines	NNP	Xx	0	I-ORG
Holding	NNP	Xx	0	I-ORG
discussed	VBD	x	0	O
the	DT	x	0	O
Chicago	NNP	Xx	1	B-LOC
route	NN	x	0	O
.	.	.	0	O

Figure 8.16 Some NER features for a sample sentence, assuming that Chicago and Villanueva are listed as locations in a gazetteer. We assume features only take on the values 0 or 1, so the first POS feature, for example, would be represented as $\mathbb{1}\{\text{POS} = \text{NNP}\}$.

8.5.3 Inference and Training for CRFs

How do we find the best tag sequence $\hat{Y}$ for a given input X ? We start with Eq. 8.22:

$$\begin{aligned}\hat{Y} &= \underset{Y \in \mathcal{Y}}{\operatorname{argmax}} P(Y|X) \\ &= \underset{Y \in \mathcal{Y}}{\operatorname{argmax}} \frac{1}{Z(X)} \exp \left(\sum_{k=1}^K w_k F_k(X, Y) \right)\end{aligned}\quad (8.27)$$

$$= \underset{Y \in \mathcal{Y}}{\operatorname{argmax}} \exp \left(\sum_{k=1}^K w_k \sum_{i=1}^n f_k(y_{i-1}, y_i, X, i) \right) \quad (8.28)$$

$$= \underset{Y \in \mathcal{Y}}{\operatorname{argmax}} \sum_{k=1}^K w_k \sum_{i=1}^n f_k(y_{i-1}, y_i, X, i) \quad (8.29)$$

$$= \underset{Y \in \mathcal{Y}}{\operatorname{argmax}} \sum_{i=1}^n \sum_{k=1}^K w_k f_k(y_{i-1}, y_i, X, i) \quad (8.30)$$

We can ignore the exp function and the denominator $Z(X)$, as we do above, because exp doesn't change the argmax, and the denominator $Z(X)$ is constant for a given observation sequence X .

How should we decode to find this optimal tag sequence $\hat{y}$? Just as with HMMs, we'll turn to the Viterbi algorithm, which works because, like the HMM, the linear-chain CRF depends at each timestep on only one previous output token y_{i-1} .

Concretely, this involves filling an $N \times T$ array with the appropriate values, maintaining backpointers as we proceed. As with HMM Viterbi, when the table is filled, we simply follow pointers back from the maximum value in the final column to retrieve the desired set of labels.

The requisite changes from HMM Viterbi have to do only with how we fill each cell. Recall from Eq. 8.19 that the recursive step of the Viterbi equation computes the Viterbi value of time t for state j as

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) a_{ij} b_j(o_t); \quad 1 \leq j \leq N, 1 < t \leq T \quad (8.31)$$

which is the HMM implementation of

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) P(s_j|s_i) P(o_t|s_j) \quad 1 \leq j \leq N, 1 < t \leq T \quad (8.32)$$

The CRF requires only a slight change to this latter formula, replacing the a and b prior and likelihood probabilities with the CRF features:

$$v_t(j) = \max_{i=1}^N v_{t-1}(i) + \sum_{k=1}^K w_k f_k(y_{t-1}, y_t, X, t) \quad 1 \leq j \leq N, 1 < t \leq T \quad (8.33)$$

Learning in CRFs relies on the same supervised learning algorithms we presented for logistic regression. Given a sequence of observations, feature functions, and corresponding outputs, we use stochastic gradient descent to train the weights to maximize the log-likelihood of the training corpus. The local nature of linear-chain CRFs means that the forward-backward algorithm introduced for HMMs in Appendix A can be extended to a CRF version that will efficiently compute the necessary derivatives. As with logistic regression, L1 or L2 regularization is important.

8.6 Evaluation of Named Entity Recognition

Part-of-speech taggers are evaluated by the standard metric of **accuracy**. Named entity recognizers are evaluated by **recall**, **precision**, and **F₁ measure**. Recall that recall is the ratio of the number of correctly labeled responses to the total that should have been labeled; precision is the ratio of the number of correctly labeled responses to the total labeled; and F -measure is the harmonic mean of the two.

To know if the difference between the F_1 scores of two NER systems is a significant difference, we use the paired bootstrap test, or the similar randomization test (Section 4.9).

For named entity tagging, the *entity* rather than the word is the unit of response. Thus in the example in Fig. 8.16, the two entities *Jane Villanueva* and *United Airlines Holding* and the non-entity *discussed* would each count as a single response.

The fact that named entity tagging has a segmentation component which is not present in tasks like text categorization or part-of-speech tagging causes some problems with evaluation. For example, a system that labeled *Jane* but not *Jane Villanueva* as a person would cause two errors, a false positive for O and a false negative for I-PER. In addition, using entities as the unit of response but words as the unit of training means that there is a mismatch between the training and test conditions.

8.7 Further Details

In this section we summarize a few remaining details of the data and models for part-of-speech tagging and NER, beginning with data. Since the algorithms we have

presented are supervised, having labeled data is essential for training and testing. A wide variety of datasets exist for part-of-speech tagging and/or NER. The Universal Dependencies (UD) dataset (de Marneffe et al., 2021) has POS tagged corpora in over a hundred languages, as do the Penn Treebanks in English, Chinese, and Arabic. OntoNotes has corpora labeled for named entities in English, Chinese, and Arabic (Hovy et al., 2006). Named entity tagged corpora are also available in particular domains, such as for biomedical (Bada et al., 2012) and literary text (Bamman et al., 2019).

8.7.1 Rule-based Methods

While machine learned (neural or CRF) sequence models are the norm in academic research, commercial approaches to NER are often based on pragmatic combinations of lists and rules, with some smaller amount of supervised machine learning (Chiticariu et al., 2013). For example in the IBM System T architecture, a user specifies declarative constraints for tagging tasks in a formal query language that includes regular expressions, dictionaries, semantic constraints, and other operators, which the system compiles into an efficient extractor (Chiticariu et al., 2018).

One common approach is to make repeated rule-based passes over a text, starting with rules with very high precision but low recall, and, in subsequent stages, using machine learning methods that take the output of the first pass into account (an approach first worked out for coreference (Lee et al., 2017a)):

1. First, use high-precision rules to tag unambiguous entity mentions.
2. Then, search for substring matches of the previously detected names.
3. Use application-specific name lists to find likely domain-specific mentions.
4. Finally, apply supervised sequence labeling techniques that use tags from previous stages as additional features.

Rule-based methods were also the earliest methods for part-of-speech tagging. Rule-based taggers like the English Constraint Grammar system (Karlsson et al. 1995, Voutilainen 1999) use a two-stage formalism invented in the 1950s and 1960s: (1) a morphological analyzer with tens of thousands of word stem entries returns all parts of speech for a word, then (2) a large set of thousands of constraints are applied to the input sentence to rule out parts of speech inconsistent with the context.

8.7.2 POS Tagging for Morphologically Rich Languages

Augmentations to tagging algorithms become necessary when dealing with languages with rich morphology like Czech, Hungarian and Turkish.

These productive word-formation processes result in a large vocabulary for these languages: a 250,000 word token corpus of Hungarian has more than twice as many word types as a similarly sized corpus of English (Oravecz and Dienes, 2002), while a 10 million word token corpus of Turkish contains four times as many word types as a similarly sized English corpus (Hakkani-Tür et al., 2002). Large vocabularies mean many unknown words, and these unknown words cause significant performance degradations in a wide variety of languages (including Czech, Slovene, Estonian, and Romanian) (Hajič, 2000).

Highly inflectional languages also have much more information than English coded in word morphology, like **case** (nominative, accusative, genitive) or **gender** (masculine, feminine). Because this information is important for tasks like parsing and coreference resolution, part-of-speech taggers for morphologically rich lan-

languages need to label words with case and gender information. Tagsets for morphologically rich languages are therefore sequences of morphological tags rather than a single primitive tag. Here's a Turkish example, in which the word *izin* has three possible morphological/part-of-speech tags and meanings (Hakkani-Tür et al., 2002):

- | | |
|---|---------------------------|
| 1. Yerdeki izin temizlenmesi gerek.
The trace on the floor should be cleaned. | iz + Noun+A3sg+Pnon+Gen |
| 2. Üzerinde parmak izin kalmış.
Your finger print is left on (it). | iz + Noun+A3sg+P2sg+Nom |
| 3. İçeri girmek için izin alman gerekiyor.
You need permission to enter. | izin + Noun+A3sg+Pnon+Nom |

Using a morphological parse sequence like Noun+A3sg+Pnon+Gen as the part-of-speech tag greatly increases the number of parts of speech, and so tagsets can be 4 to 10 times larger than the 50–100 tags we have seen for English. With such large tagsets, each word needs to be morphologically analyzed to generate the list of possible morphological tag sequences (part-of-speech tags) for the word. The role of the tagger is then to disambiguate among these tags. This method also helps with unknown words since morphological parsers can accept unknown stems and still segment the affixes properly.

8.8 Summary

This chapter introduced **parts of speech** and **named entities**, and the tasks of **part-of-speech tagging** and **named entity recognition**:

- Languages generally have a small set of **closed class** words that are highly frequent, ambiguous, and act as **function words**, and **open-class** words like **nouns, verbs, adjectives**. Various part-of-speech **tagsets** exist, of between 40 and 200 tags.
- **Part-of-speech tagging** is the process of assigning a part-of-speech label to each of a sequence of words.
- **Named entities** are words for proper nouns referring mainly to people, places, and organizations, but extended to many other types that aren't strictly entities or even proper nouns.
- Two common approaches to **sequence modeling** are a **generative** approach, **HMM** tagging, and a **discriminative** approach, **CRF** tagging. We will see a neural approach in following chapters.
- The probabilities in HMM taggers are estimated by maximum likelihood estimation on tag-labeled training corpora. The Viterbi algorithm is used for **decoding**, finding the most likely tag sequence
- **Conditional Random Fields** or **CRF taggers** train a log-linear model that can choose the best tag sequence given an observation sequence, based on features that condition on the output tag, the prior output tag, the entire input sequence, and the current timestep. They use the Viterbi algorithm for inference, to choose the best sequence of tags, and a version of the Forward-Backward algorithm (see Appendix A) for training,

Bibliographical and Historical Notes

What is probably the earliest part-of-speech tagger was part of the parser in Zellig Harris's Transformations and Discourse Analysis Project (TDAP), implemented between June 1958 and July 1959 at the University of Pennsylvania (Harris, 1962), although earlier systems had used part-of-speech dictionaries. TDAP used 14 handwritten rules for part-of-speech disambiguation; the use of part-of-speech tag sequences and the relative frequency of tags for a word prefigures modern algorithms. The parser was implemented essentially as a cascade of finite-state transducers; see Joshi and Hopely (1999) and Karttunen (1999) for a reimplementa-

The Computational Grammar Coder (CGC) of Klein and Simmons (1963) had three components: a lexicon, a morphological analyzer, and a context disambiguator. The small 1500-word lexicon listed only function words and other irregular words. The morphological analyzer used inflectional and derivational suffixes to assign part-of-speech classes. These were run over words to produce candidate parts of speech which were then disambiguated by a set of 500 context rules by relying on surrounding islands of unambiguous words. For example, one rule said that between an ARTICLE and a VERB, the only allowable sequences were ADJ-NOUN, NOUN-ADVERB, or NOUN-NOUN. The TAGGIT tagger (Greene and Rubin, 1971) used the same architecture as Klein and Simmons (1963), with a bigger dictionary and more tags (87). TAGGIT was applied to the Brown corpus and, according to Francis and Kučera (1982, p. 9), accurately tagged 77% of the corpus; the remainder of the Brown corpus was then tagged by hand. All these early algorithms were based on a two-stage architecture in which a dictionary was first used to assign each word a set of potential parts of speech, and then lists of handwritten disambiguation rules winnowed the set down to a single part of speech per word.

Probabilities were used in tagging by Stolz et al. (1965) and a complete probabilistic tagger with Viterbi decoding was sketched by Bahl and Mercer (1976). The Lancaster-Oslo/Bergen (LOB) corpus, a British English equivalent of the Brown corpus, was tagged in the early 1980's with the CLAWS tagger (Marshall 1983; Marshall 1987; Garside 1987), a probabilistic algorithm that approximated a simplified HMM tagger. The algorithm used tag bigram probabilities, but instead of storing the word likelihood of each tag, the algorithm marked tags either as *rare* ($P(\text{tag}|\text{word}) < .01$) *infrequent* ($P(\text{tag}|\text{word}) < .10$) or *normally frequent* ($P(\text{tag}|\text{word}) > .10$).

DeRose (1988) developed a quasi-HMM algorithm, including the use of dynamic programming, although computing $P(t|w)P(w)$ instead of $P(w|t)P(w)$. The same year, the probabilistic PARTS tagger of Church 1988, 1989 was probably the first implemented HMM tagger, described correctly in Church (1989), although Church (1988) also described the computation incorrectly as $P(t|w)P(w)$ instead of $P(w|t)P(w)$. Church (p.c.) explained that he had simplified for pedagogical purposes because using the probability $P(t|w)$ made the idea seem more understandable as “storing a lexicon in an almost standard form”.

Later taggers explicitly introduced the use of the hidden Markov model (Kupiec 1992; Weischedel et al. 1993; Schütze and Singer 1994). Merialdo (1994) showed that fully unsupervised EM didn't work well for the tagging task and that reliance on hand-labeled data was important. Charniak et al. (1993) showed the importance of the most frequent tag baseline; the 92.3% number we give above was from Abney et al. (1999). See Brants (2000) for HMM tagger implementation details, including the extension to trigram contexts, and the use of sophisticated unknown word features; its performance is still close to state of the art taggers.

Log-linear models for POS tagging were introduced by [Ratnaparkhi \(1996\)](#), who introduced a system called MXPOST which implemented a maximum entropy Markov model (MEMM), a slightly simpler version of a CRF. Around the same time, sequence labelers were applied to the task of named entity tagging, first with HMMs ([Bikel et al., 1997](#)) and MEMMs ([McCallum et al., 2000](#)), and then once CRFs were developed ([Lafferty et al. 2001](#)), they were also applied to NER ([McCallum and Li, 2003](#)). A wide exploration of features followed ([Zhou et al., 2005](#)). Neural approaches to NER mainly follow from the pioneering results of [Collobert et al. \(2011\)](#), who applied a CRF on top of a convolutional net. BiLSTMs with word and character-based embeddings as input followed shortly and became a standard neural algorithm for NER ([Huang et al. 2015](#), [Ma and Hovy 2016](#), [Lample et al. 2016](#)) followed by the more recent use of Transformers and BERT.

The idea of using letter suffixes for unknown words is quite old; the early [Klein and Simmons \(1963\)](#) system checked all final letter suffixes of lengths 1-5. The unknown word features described on page 176 come mainly from [Ratnaparkhi \(1996\)](#), with augmentations from [Toutanova et al. \(2003\)](#) and [Manning \(2011\)](#).

State of the art POS taggers use neural algorithms, either bidirectional RNNs or Transformers like BERT; see Chapter 9 and Chapter 11. HMM ([Brants 2000](#); [Thede and Harper 1999](#)) and CRF tagger accuracies are likely just a tad lower.

[Manning \(2011\)](#) investigates the remaining 2.7% of errors in a high-performing tagger ([Toutanova et al., 2003](#)). He suggests that a third or half of these remaining errors are due to errors or inconsistencies in the training data, a third might be solvable with richer linguistic models, and for the remainder the task is underspecified or unclear.

Supervised tagging relies heavily on in-domain training data hand-labeled by experts. Ways to relax this assumption include unsupervised algorithms for clustering words into part-of-speech-like classes, summarized in [Christodoulopoulos et al. \(2010\)](#), and ways to combine labeled and unlabeled data, for example by co-training ([Clark et al. 2003](#); [Søgaard 2010](#)).

See [Householder \(1995\)](#) for historical notes on parts of speech, and [Sampson \(1987\)](#) and [Garside et al. \(1997\)](#) on the provenance of the Brown and other tagsets.

Exercises

- 8.1** Find one tagging error in each of the following sentences that are tagged with the Penn Treebank tagset:
1. I/PRP need/VBP a/DT flight/NN from/IN Atlanta/NN
 2. Does/VBZ this/DT flight/NN serve/VB dinner/NNS
 3. I/PRP have/VB a/DT friend/NN living/VBG in/IN Denver/NNP
 4. Can/VBP you/PRP list/VB the/DT nonstop/JJ afternoon/NN flights/NNS
- 8.2** Use the Penn Treebank tagset to tag each word in the following sentences from Damon Runyon's short stories. You may ignore punctuation. Some of these are quite difficult; do your best.
1. It is a nice night.
 2. This crap game is over a garage in Fifty-second Street. . .
 3. . . .Nobody ever takes the newspapers she sells . . .
 4. He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.

5. ... I am sitting in Mindy's restaurant putting on the gefillte fish, which is a dish I am very fond of, ...
 6. When a guy and a doll get to taking peeks back and forth at each other, why there you are indeed.
- 8.3** Now compare your tags from the previous exercise with one or two friend's answers. On which words did you disagree the most? Why?
 - 8.4** Implement the "most likely tag" baseline. Find a POS-tagged training set, and use it to compute for each word the tag that maximizes $p(t|w)$. You will need to implement a simple tokenizer to deal with sentence boundaries. Start by assuming that all unknown words are NN and compute your error rate on known and unknown words. Now write at least five rules to do a better job of tagging unknown words, and show the difference in error rates.
 - 8.5** Build a bigram HMM tagger. You will need a part-of-speech-tagged corpus. First split the corpus into a training set and test set. From the labeled training set, train the transition and observation probabilities of the HMM tagger directly on the hand-tagged data. Then implement the Viterbi algorithm so you can decode a test sentence. Now run your algorithm on the test set. Report its error rate and compare its performance to the most frequent tag baseline.
 - 8.6** Do an error analysis of your tagger. Build a confusion matrix and investigate the most frequent errors. Propose some features for improving the performance of your tagger on these errors.
 - 8.7** Develop a set of regular expressions to recognize the character shape features described on page 176.
 - 8.8** The BIO and other labeling schemes given in this chapter aren't the only possible one. For example, the B tag can be reserved only for those situations where an ambiguity exists between adjacent entities. Propose a new set of BIO tags for use with your NER system. Experiment with it and compare its performance with the schemes presented in this chapter.
 - 8.9** Names of works of art (books, movies, video games, etc.) are quite different from the kinds of named entities we've discussed in this chapter. Collect a list of names of works of art from a particular category from a Web-based source (e.g., gutenberg.org, amazon.com, imdb.com, etc.). Analyze your list and give examples of ways that the names in it are likely to be problematic for the techniques described in this chapter.
 - 8.10** Develop an NER system specific to the category of names that you collected in the last exercise. Evaluate your system on a collection of text likely to contain instances of these named entities.

RNNs and LSTMs

Time will explain.
Jane Austen, *Persuasion*

Language is an inherently temporal phenomenon. Spoken language is a sequence of acoustic events over time, and we comprehend and produce both spoken and written language as a continuous input stream. The temporal nature of language is reflected in the metaphors we use; we talk of the *flow of conversations*, *news feeds*, and *twitter streams*, all of which emphasize that language is a sequence that unfolds in time.

This temporal nature is reflected in some language processing algorithms. For example, the Viterbi algorithm we introduced for HMM part-of-speech tagging proceeds through the input a word at a time, carrying forward information gleaned along the way. Yet other machine learning approaches, like those we've studied for sentiment analysis or other text classification tasks don't have this temporal nature – they assume simultaneous access to all aspects of their input.

The feedforward networks of Chapter 7 also assumed simultaneous access, although they also had a simple model for time. Recall that we applied feedforward networks to language modeling by having them look only at a fixed-size window of words, and then sliding this window over the input, making independent predictions along the way. This sliding-window approach is also used in the transformer architecture we will introduce in Chapter 10.

This chapter introduces a deep learning architecture that offers an alternative way of representing time: recurrent neural networks (RNNs), and their variants like LSTMs. RNNs have a mechanism that deals directly with the sequential nature of language, allowing them to handle the temporal nature of language without the use of arbitrary fixed-sized windows. The recurrent network offers a new way to represent the prior context, in its **recurrent connections**, allowing the model's decision to depend on information from hundreds of words in the past. We'll see how to apply the model to the task of language modeling, to sequence modeling tasks like part-of-speech tagging, and to text classification tasks like sentiment analysis.

9.1 Recurrent Neural Networks

A recurrent neural network (RNN) is any network that contains a cycle within its network connections, meaning that the value of some unit is directly, or indirectly, dependent on its own earlier outputs as an input. While powerful, such networks are difficult to reason about and to train. However, within the general class of recurrent networks there are constrained architectures that have proven to be extremely effective when applied to language. In this section, we consider a class of recurrent networks referred to as **Elman Networks** (Elman, 1990) or **simple recurrent net-**

works. These networks are useful in their own right and serve as the basis for more complex approaches like the Long Short-Term Memory (LSTM) networks discussed later in this chapter. In this chapter when we use the term RNN we'll be referring to these simpler more constrained networks (although you will often see the term RNN to mean any net with recurrent properties including LSTMs).

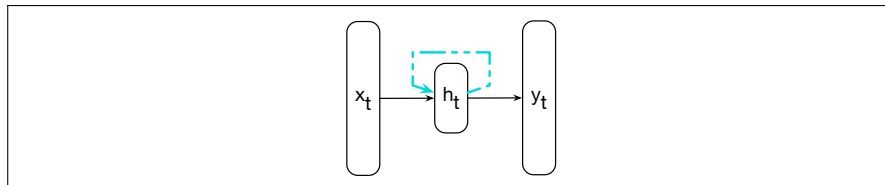


Figure 9.1 Simple recurrent neural network after [Elman \(1990\)](#). The hidden layer includes a recurrent connection as part of its input. That is, the activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.

Fig. 9.1 illustrates the structure of an RNN. As with ordinary feedforward networks, an input vector representing the current input, $\mathbf{x}_t$, is multiplied by a weight matrix and then passed through a non-linear activation function to compute the values for a layer of hidden units. This hidden layer is then used to calculate a corresponding output, $\mathbf{y}_t$. In a departure from our earlier window-based approach, sequences are processed by presenting one item at a time to the network. We'll use subscripts to represent time, thus $\mathbf{x}_t$ will mean the input vector $\mathbf{x}$ at time t . The key difference from a feedforward network lies in the recurrent link shown in the figure with the dashed line. This link augments the input to the computation at the hidden layer with the value of the hidden layer *from the preceding point in time*.

The hidden layer from the previous time step provides a form of memory, or context, that encodes earlier processing and informs the decisions to be made at later points in time. Critically, this approach does not impose a fixed-length limit on this prior context; the context embodied in the previous hidden layer can include information extending back to the beginning of the sequence.

Adding this temporal dimension makes RNNs appear to be more complex than non-recurrent architectures. But in reality, they're not all that different. Given an input vector and the values for the hidden layer from the previous time step, we're still performing the standard feedforward calculation introduced in Chapter 7. To see this, consider Fig. 9.2 which clarifies the nature of the recurrence and how it factors into the computation at the hidden layer. The most significant change lies in the new set of weights, $\mathbf{U}$, that connect the hidden layer from the previous time step to the current hidden layer. These weights determine how the network makes use of past context in calculating the output for the current input. As with the other weights in the network, these connections are trained via backpropagation.

9.1.1 Inference in RNNs

Forward inference (mapping a sequence of inputs to a sequence of outputs) in an RNN is nearly identical to what we've already seen with feedforward networks. To compute an output $\mathbf{y}_t$ for an input $\mathbf{x}_t$, we need the activation value for the hidden layer $\mathbf{h}_t$. To calculate this, we multiply the input $\mathbf{x}_t$ with the weight matrix $\mathbf{W}$, and the hidden layer from the previous time step $\mathbf{h}_{t-1}$ with the weight matrix $\mathbf{U}$. We add these values together and pass them through a suitable activation function, g , to arrive at the activation value for the current hidden layer, $\mathbf{h}_t$. Once we have the

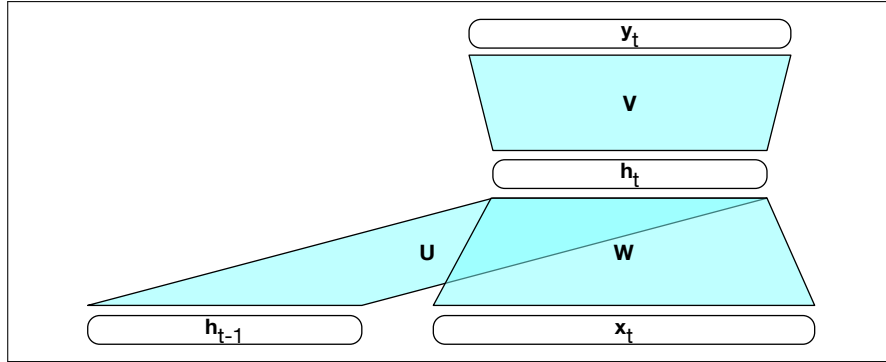


Figure 9.2 Simple recurrent neural network illustrated as a feedforward network.

values for the hidden layer, we proceed with the usual computation to generate the output vector.

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t) \quad (9.1)$$

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t) \quad (9.2)$$

It's worthwhile here to be careful about specifying the dimensions of the input, hidden and output layers, as well as the weight matrices to make sure these calculations are correct. Let's refer to the input, hidden and output layer dimensions as d_{in} , d_h , and d_{out} respectively. Given this, our three parameter matrices are: $\mathbf{W} \in \mathbb{R}^{d_h \times d_{in}}$, $\mathbf{U} \in \mathbb{R}^{d_h \times d_h}$, and $\mathbf{V} \in \mathbb{R}^{d_{out} \times d_h}$.

In the commonly encountered case of soft classification, computing $\mathbf{y}_t$ consists of a softmax computation that provides a probability distribution over the possible output classes.

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) \quad (9.3)$$

The fact that the computation at time t requires the value of the hidden layer from time $t - 1$ mandates an incremental inference algorithm that proceeds from the start of the sequence to the end as illustrated in Fig. 9.3. The sequential nature of simple recurrent networks can also be seen by *unrolling* the network in time as is shown in Fig. 9.4. In this figure, the various layers of units are copied for each time step to illustrate that they will have differing values over time. However, the various weight matrices are shared across time.

```

function FORWARDRNN(x, network) returns output sequence y

  h0 ← 0
  for i ← 1 to LENGTH(x) do
    hi ← g(Uhi-1 + Wxi)
    yi ← f(Vhi)
  return y

```

Figure 9.3 Forward inference in a simple recurrent network. The matrices $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$ are shared across time, while new values for $\mathbf{h}$ and $\mathbf{y}$ are calculated with each time step.

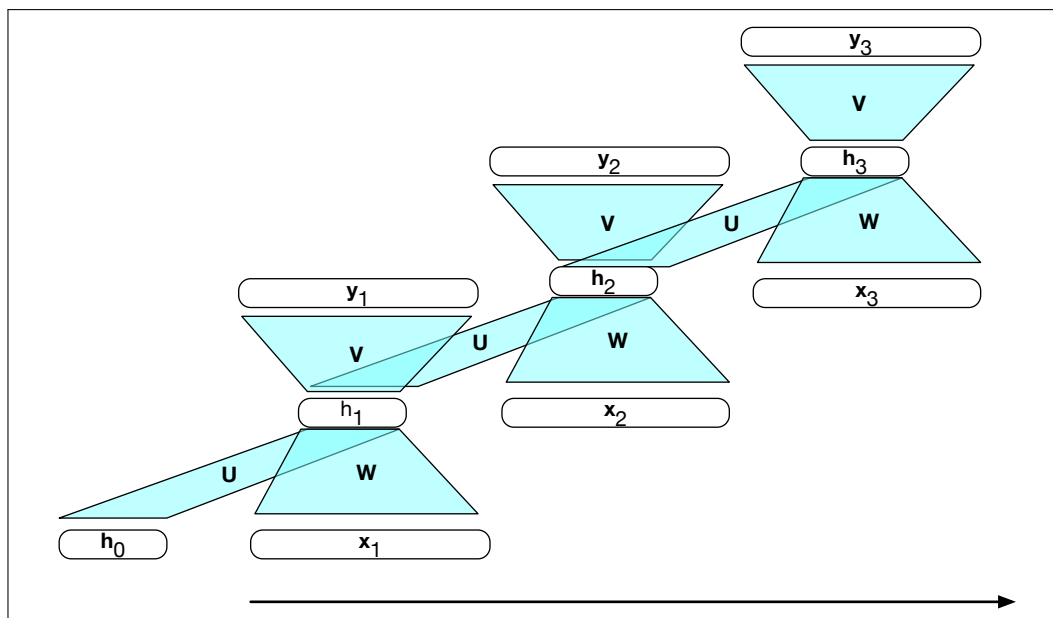


Figure 9.4 A simple recurrent neural network shown unrolled in time. Network layers are recalculated for each time step, while the weights U , V and W are shared across all time steps.

9.1.2 Training

As with feedforward networks, we'll use a training set, a loss function, and backpropagation to obtain the gradients needed to adjust the weights in these recurrent networks. As shown in Fig. 9.2, we now have 3 sets of weights to update: W , the weights from the input layer to the hidden layer, U , the weights from the previous hidden layer to the current hidden layer, and finally V , the weights from the hidden layer to the output layer.

Fig. 9.4 highlights two considerations that we didn't have to worry about with backpropagation in feedforward networks. First, to compute the loss function for the output at time t we need the hidden layer from time $t - 1$. Second, the hidden layer at time t influences both the output at time t and the hidden layer at time $t + 1$ (and hence the output and loss at $t + 1$). It follows from this that to assess the error accruing to h_t , we'll need to know its influence on both the current output *as well as the ones that follow*.

Tailoring the backpropagation algorithm to this situation leads to a two-pass algorithm for training the weights in RNNs. In the first pass, we perform forward inference, computing h_t, y_t , accumulating the loss at each step in time, saving the value of the hidden layer at each step for use at the next time step. In the second phase, we process the sequence in reverse, computing the required gradients as we go, computing and saving the error term for use in the hidden layer for each step backward in time. This general approach is commonly referred to as **backpropagation through time** (Werbos 1974, Rumelhart et al. 1986, Werbos 1990).

Fortunately, with modern computational frameworks and adequate computing resources, there is no need for a specialized approach to training RNNs. As illustrated in Fig. 9.4, explicitly unrolling a recurrent network into a feedforward computational graph eliminates any explicit recurrences, allowing the network weights to be trained directly. In such an approach, we provide a template that specifies the basic structure of the network, including all the necessary parameters for the input,

backpropagation
through time

output, and hidden layers, the weight matrices, as well as the activation and output functions to be used. Then, when presented with a specific input sequence, we can generate an unrolled feedforward network specific to that input, and use that graph to perform forward inference or training via ordinary backpropagation.

For applications that involve much longer input sequences, such as speech recognition, character-level processing, or streaming continuous inputs, unrolling an entire input sequence may not be feasible. In these cases, we can unroll the input into manageable fixed-length segments and treat each segment as a distinct training item.

9.2 RNNs as Language Models

Let's see how to apply RNNs to the language modeling task. Recall from Chapter 3 that language models predict the next word in a sequence given some preceding context. For example, if the preceding context is “*Thanks for all the*” and we want to know how likely the next word is “*fish*” we would compute:

$$P(\textit{fish}|\textit{Thanks for all the})$$

Language models give us the ability to assign such a conditional probability to every possible next word, giving us a distribution over the entire vocabulary. We can also assign probabilities to entire sequences by combining these conditional probabilities with the chain rule:

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i|w_{<i})$$

The n -gram language models of Chapter 3 compute the probability of a word given counts of its occurrence with the $n - 1$ prior words. The context is thus of size $n - 1$. For the feedforward language models of Chapter 7, the context is the window size.

RNN language models (Mikolov et al., 2010) process the input sequence one word at a time, attempting to predict the next word from the current word and the previous hidden state. RNNs thus don't have the limited context problem that n -gram models have, or the fixed context that feedforward language models have, since the hidden state can in principle represent information about all of the preceding words all the way back to the beginning of the sequence. Fig. 9.5 sketches this difference between a FFN language model and an RNN language model, showing that the RNN language model uses h_{t-1} , the hidden state from the previous time step, as a representation of the past context.

9.2.1 Forward Inference in an RNN language model

Forward inference in a recurrent language model proceeds exactly as described in Section 9.1.1. The input sequence $\mathbf{X} = [\mathbf{x}_1; \dots; \mathbf{x}_t; \dots; \mathbf{x}_N]$ consists of a series of word embeddings each represented as a one-hot vector of size $|V| \times 1$, and the output prediction, $\mathbf{y}$, is a vector representing a probability distribution over the vocabulary. At each step, the model uses the word embedding matrix $\mathbf{E}$ to retrieve the embedding for the current word, and then combines it with the hidden layer from the previous step to compute a new hidden layer. This hidden layer is then used to generate an output layer which is passed through a softmax layer to generate a probability

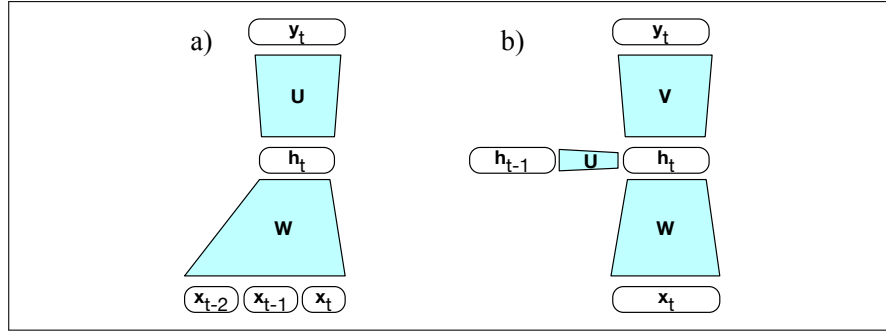


Figure 9.5 Simplified sketch of (a) a feedforward neural language model versus (b) an RNN language model moving through a text.

distribution over the entire vocabulary. That is, at time t :

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t \quad (9.4)$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \quad (9.5)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t) \quad (9.6)$$

The vector resulting from $\mathbf{V}\mathbf{h}$ can be thought of as a set of scores over the vocabulary given the evidence provided in $\mathbf{h}$. Passing these scores through the softmax normalizes the scores into a probability distribution. The probability that a particular word i in the vocabulary is the next word is represented by $\mathbf{y}_t[i]$, the i th component of $\mathbf{y}_t$:

$$P(w_{t+1} = i | w_1, \dots, w_t) = \mathbf{y}_t[i] \quad (9.7)$$

The probability of an entire sequence is just the product of the probabilities of each item in the sequence, where we'll use $\mathbf{y}_i[w_i]$ to mean the probability of the true word w_i at time step i .

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1}) \quad (9.8)$$

$$= \prod_{i=1}^n \mathbf{y}_i[w_i] \quad (9.9)$$

9.2.2 Training an RNN language model

self-supervision

To train an RNN as a language model, we use the same **self-supervision** algorithm we saw earlier: we take a corpus of text as training material and at each time step t we have the model predict the next word. We call such a model self-supervised because we don't have to add any special gold labels to the data; the natural sequence of words is its own supervision! We simply train the model to minimize the error in predicting the true next word in the training sequence, using cross-entropy as the loss function. Recall that the cross-entropy loss measures the difference between a predicted probability distribution and the correct distribution.

$$L_{CE} = - \sum_{w \in V} \mathbf{y}_t[w] \log \hat{\mathbf{y}}_t[w] \quad (9.10)$$

In the case of language modeling, the correct distribution $\mathbf{y}_t$ comes from knowing the next word. This is represented as a one-hot vector corresponding to the vocabulary

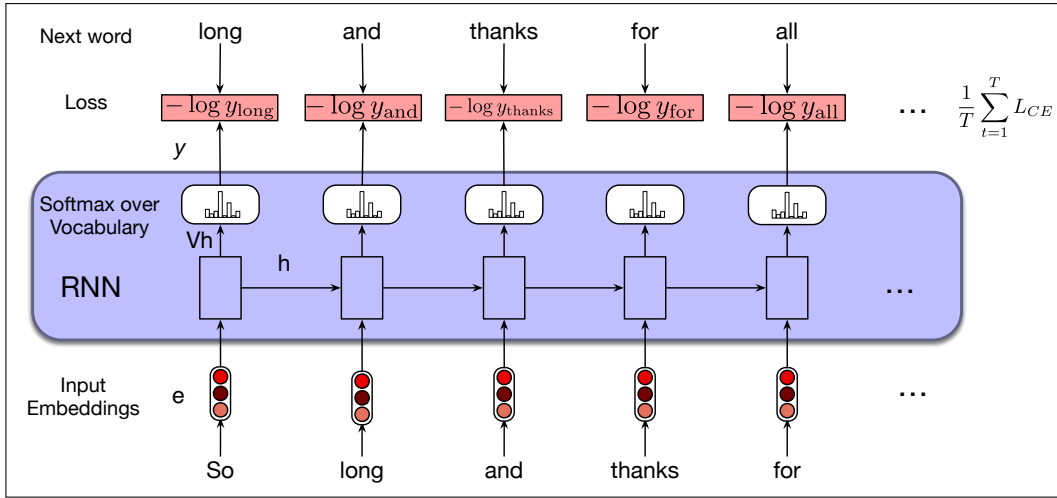


Figure 9.6 Training RNNs as language models.

where the entry for the actual next word is 1, and all the other entries are 0. Thus, the cross-entropy loss for language modeling is determined by the probability the model assigns to the correct next word. So at time t the CE loss is the negative log probability the model assigns to the next word in the training sequence.

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}] \quad (9.11)$$

Thus at each word position t of the input, the model takes as input the correct sequence of tokens $w_{1:t}$, and uses them to compute a probability distribution over possible next words so as to compute the model's loss for the next token w_{t+1} . Then we move to the next word, we ignore what the model predicted for the next word and instead use the correct sequence of tokens $w_{1:t+1}$ to estimate the probability of token w_{t+2} . This idea that we always give the model the correct history sequence to predict the next word (rather than feeding the model its best case from the previous time step) is called **teacher forcing**.

teacher forcing

The weights in the network are adjusted to minimize the average CE loss over the training sequence via gradient descent. Fig. 9.6 illustrates this training regimen.

9.2.3 Weight Tying

Careful readers may have noticed that the input embedding matrix $\mathbf{E}$ and the final layer matrix $\mathbf{V}$, which feeds the output softmax, are quite similar. The columns of $\mathbf{E}$ represent the word embeddings for each word in the vocabulary learned during the training process with the goal that words that have similar meaning and function will have similar embeddings. And, since the length of these embeddings corresponds to the size of the hidden layer d_h , the shape of the embedding matrix $\mathbf{E}$ is $d_h \times |V|$.

The final layer matrix $\mathbf{V}$ provides a way to score the likelihood of each word in the vocabulary given the evidence present in the final hidden layer of the network through the calculation of $\mathbf{Vh}$. This results in dimensionality $|V| \times d_h$. That is, the rows of $\mathbf{V}$ provide a *second set* of learned word embeddings that capture relevant aspects of word meaning. This leads to an obvious question – is it even necessary to have both? **Weight tying** is a method that dispenses with this redundancy and simply uses a single set of embeddings at the input and softmax layers. That is, we

Weight tying

dispense with V and use E in both the start and end of the computation.

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t \quad (9.12)$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t) \quad (9.13)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{E}^T \mathbf{h}_t) \quad (9.14)$$

In addition to providing improved model perplexity, this approach significantly reduces the number of parameters required for the model.

9.3 RNNs for other NLP tasks

Now that we've seen the basic RNN architecture, let's consider how to apply it to three types of NLP tasks: *sequence classification* tasks like sentiment analysis and topic classification, *sequence labeling* tasks like part-of-speech tagging, and *text generation* tasks, including with a new architecture called the **encoder-decoder**.

9.3.1 Sequence Labeling

In sequence labeling, the network's task is to assign a label chosen from a small fixed set of labels to each element of a sequence, like the part-of-speech tagging and named entity recognition tasks from Chapter 8. In an RNN approach to sequence labeling, inputs are word embeddings and the outputs are tag probabilities generated by a softmax layer over the given tagset, as illustrated in Fig. 9.7.

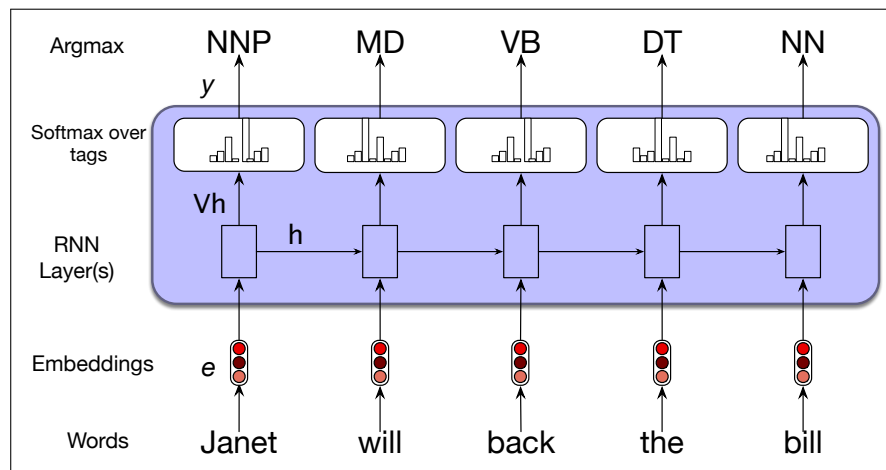


Figure 9.7 Part-of-speech tagging as sequence labeling with a simple RNN. Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.

In this figure, the inputs at each time step are pre-trained word embeddings corresponding to the input tokens. The RNN block is an abstraction that represents an unrolled simple recurrent network consisting of an input layer, hidden layer, and output layer at each time step, as well as the shared $\mathbf{U}$, $\mathbf{V}$ and $\mathbf{W}$ weight matrices that comprise the network. The outputs of the network at each time step represent the distribution over the POS tagset generated by a softmax layer.

To generate a sequence of tags for a given input, we run forward inference over the input sequence and select the most likely tag from the softmax at each step. Since we're using a softmax layer to generate the probability distribution over the output tagset at each time step, we will again employ the cross-entropy loss during training.

9.3.2 RNNs for Sequence Classification

Another use of RNNs is to classify entire sequences rather than the tokens within them. This is the set of tasks commonly called **text classification**, like sentiment analysis or spam detection, in which we classify a text into two or three classes (like positive or negative), as well as classification tasks with a large number of categories, like document-level topic classification, or message routing for customer service applications.

To apply RNNs in this setting, we pass the text to be classified through the RNN a word at a time generating a new hidden layer at each time step. We can then take the hidden layer for the last token of the text, h_n , to constitute a compressed representation of the entire sequence. We can pass this representation h_n to a feedforward network that chooses a class via a softmax over the possible classes. Fig. 9.8 illustrates this approach.

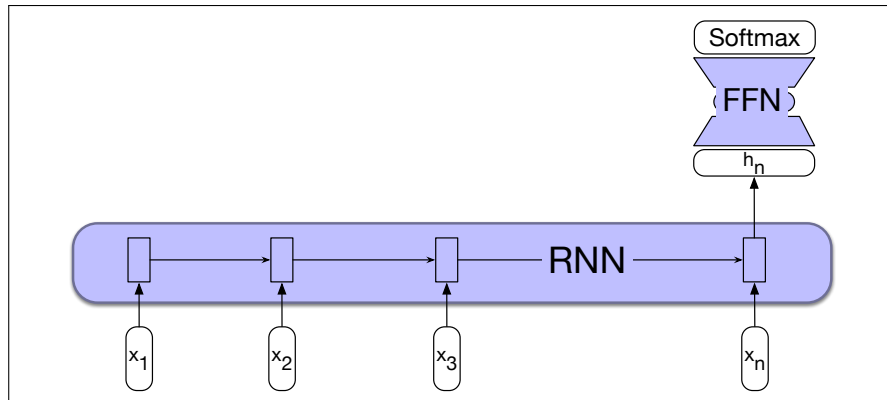


Figure 9.8 Sequence classification using a simple RNN combined with a feedforward network. The final hidden state from the RNN is used as the input to a feedforward network that performs the classification.

Note that in this approach we don't need intermediate outputs for the words in the sequence preceding the last element. Therefore, there are no loss terms associated with those elements. Instead, the loss function used to train the weights in the network is based entirely on the final text classification task. The output from the softmax output from the feedforward classifier together with a cross-entropy loss drives the training. The error signal from the classification is backpropagated all the way through the weights in the feedforward classifier through, to its input, and then through to the three sets of weights in the RNN as described earlier in Section 9.1.2. The training regimen that uses the loss from a downstream application to adjust the weights all the way through the network is referred to as **end-to-end training**.

end-to-end
training

pooling

Another option, instead of using just the last token h_n to represent the whole sequence, is to use some sort of **pooling** function of all the hidden states h_i for each word i in the sequence. For example, we can create a representation that pools all

the n hidden states by taking their element-wise mean:

$$\mathbf{h}_{mean} = \frac{1}{n} \sum_{i=1}^n \mathbf{h}_i \quad (9.15)$$

Or we can take the element-wise max; the element-wise max of a set of n vectors is a new vector whose k th element is the max of the k th elements of all the n vectors.

9.3.3 Generation with RNN-Based Language Models

RNN-based language models can also be used to generate text. Text generation is of enormous practical importance, part of tasks like question answering, machine translation, text summarization, grammar correction, story generation, and conversational dialogue; any task where a system needs to produce text, conditioned on some other text. This use of a language model to generate text is one of the areas in which the impact of neural language models on NLP has been the largest. Text generation, along with image generation and code generation, constitute a new area of AI that is often called **generative AI**.

Recall back in Chapter 3 we saw how to generate text from an n -gram language model by adapting a **sampling** technique suggested at about the same time by Claude Shannon (Shannon, 1951) and the psychologists George Miller and Jennifer Selfridge (Miller and Selfridge, 1950). We first randomly sample a word to begin a sequence based on its suitability as the start of a sequence. We then continue to sample words *conditioned on our previous choices* until we reach a pre-determined length, or an end of sequence token is generated.

autoregressive
generation

Today, this approach of using a language model to incrementally generate words by repeatedly sampling the next word conditioned on our previous choices is called **autoregressive generation** or **causal LM generation**. The procedure is basically the same as that described on page 40, but adapted to a neural context:

- Sample a word in the output from the softmax distribution that results from using the beginning of sentence marker, $\langle s \rangle$, as the first input.
- Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion.
- Continue generating until the end of sentence marker, $\langle /s \rangle$, is sampled or a fixed length limit is reached.

Technically an **autoregressive** model is a model that predicts a value at time t based on a linear function of the previous values at times $t - 1$, $t - 2$, and so on. Although language models are not linear (since they have many layers of non-linearities), we loosely refer to this generation technique as **autoregressive generation** since the word generated at each time step is conditioned on the word selected by the network from the previous step. Fig. 9.9 illustrates this approach. In this figure, the details of the RNN's hidden layers and recurrent connections are hidden within the blue block.

This simple architecture underlies state-of-the-art approaches to applications such as machine translation, summarization, and question answering. The key to these approaches is to prime the generation component with an appropriate context. That is, instead of simply using $\langle s \rangle$ to get things started we can provide a richer task-appropriate context; for translation the context is the sentence in the source language; for summarization it's the long text we want to summarize.

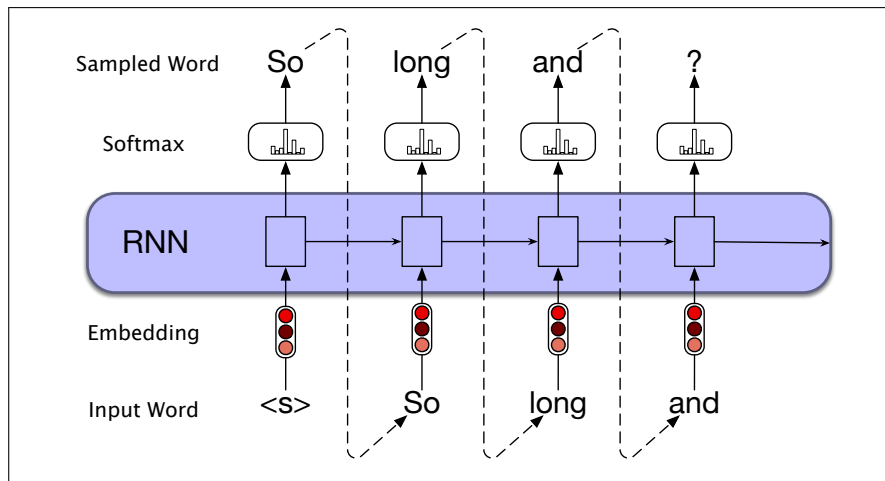


Figure 9.9 Autoregressive generation with an RNN-based neural language model.

9.4 Stacked and Bidirectional RNN architectures

Recurrent networks are quite flexible. By combining the feedforward nature of unrolled computational graphs with vectors as common inputs and outputs, complex networks can be treated as modules that can be combined in creative ways. This section introduces two of the more common network architectures used in language processing with RNNs.

9.4.1 Stacked RNNs

In our examples thus far, the inputs to our RNNs have consisted of sequences of word or character embeddings (vectors) and the outputs have been vectors useful for predicting words, tags or sequence labels. However, nothing prevents us from using the entire sequence of outputs from one RNN as an input sequence to another one. **Stacked RNNs** consist of multiple networks where the output of one layer serves as the input to a subsequent layer, as shown in Fig. 9.10.

Stacked RNNs

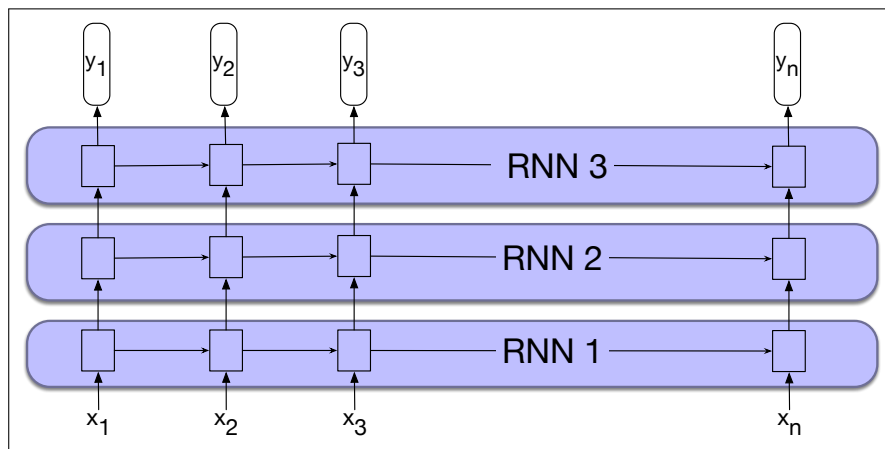


Figure 9.10 Stacked recurrent networks. The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output.

Stacked RNNs generally outperform single-layer networks. One reason for this success seems to be that the network induces representations at differing levels of abstraction across layers. Just as the early stages of the human visual system detect edges that are then used for finding larger regions and shapes, the initial layers of stacked networks can induce representations that serve as useful abstractions for further layers—representations that might prove difficult to induce in a single RNN. The optimal number of stacked RNNs is specific to each application and to each training set. However, as the number of stacks is increased the training costs rise quickly.

9.4.2 Bidirectional RNNs

The RNN uses information from the left (prior) context to make its predictions at time t . But in many applications we have access to the entire input sequence; in those cases we would like to use words from the context to the right of t . One way to do this is to run two separate RNNs, one left-to-right, and one right-to-left, and concatenate their representations.

In the left-to-right RNNs we've discussed so far, the hidden state at a given time t represents everything the network knows about the sequence up to that point. The state is a function of the inputs $x_1, \dots, x_t$ and represents the context of the network to the left of the current time.

$$\mathbf{h}_t^f = \text{RNN}_{\text{forward}}(\mathbf{x}_1, \dots, \mathbf{x}_t) \quad (9.16)$$

This new notation $\mathbf{h}_t^f$ simply corresponds to the normal hidden state at time t , representing everything the network has gleaned from the sequence so far.

To take advantage of context to the right of the current input, we can train an RNN on a *reversed* input sequence. With this approach, the hidden state at time t represents information about the sequence to the *right* of the current input:

$$\mathbf{h}_t^b = \text{RNN}_{\text{backward}}(\mathbf{x}_t, \dots, \mathbf{x}_n) \quad (9.17)$$

Here, the hidden state $\mathbf{h}_t^b$ represents all the information we have discerned about the sequence from t to the end of the sequence.

bidirectional
RNN

A **bidirectional RNN** (Schuster and Paliwal, 1997) combines two independent RNNs, one where the input is processed from the start to the end, and the other from the end to the start. We then concatenate the two representations computed by the networks into a single vector that captures both the left and right contexts of an input at each point in time. Here we use either the semicolon ";" or the equivalent symbol $\oplus$ to mean vector concatenation:

$$\begin{aligned} \mathbf{h}_t &= [\mathbf{h}_t^f ; \mathbf{h}_t^b] \\ &= \mathbf{h}_t^f \oplus \mathbf{h}_t^b \end{aligned} \quad (9.18)$$

Fig. 9.11 illustrates such a bidirectional network that concatenates the outputs of the forward and backward pass. Other simple ways to combine the forward and backward contexts include element-wise addition or multiplication. The output at each step in time thus captures information to the left and to the right of the current input. In sequence labeling applications, these concatenated outputs can serve as the basis for a local labeling decision.

Bidirectional RNNs have also proven to be quite effective for sequence classification. Recall from Fig. 9.8 that for sequence classification we used the final hidden

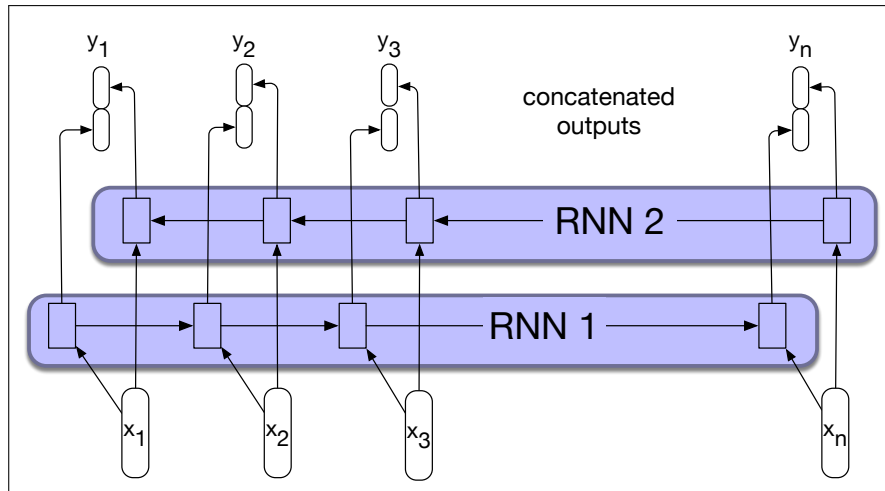


Figure 9.11 A bidirectional RNN. Separate models are trained in the forward and backward directions, with the output of each model at each time point concatenated to represent the bidirectional state at that time point.

state of the RNN as the input to a subsequent feedforward classifier. A difficulty with this approach is that the final state naturally reflects more information about the end of the sentence than its beginning. Bidirectional RNNs provide a simple solution to this problem; as shown in Fig. 9.12, we simply combine the final hidden states from the forward and backward passes (for example by concatenation) and use that as input for follow-on processing.

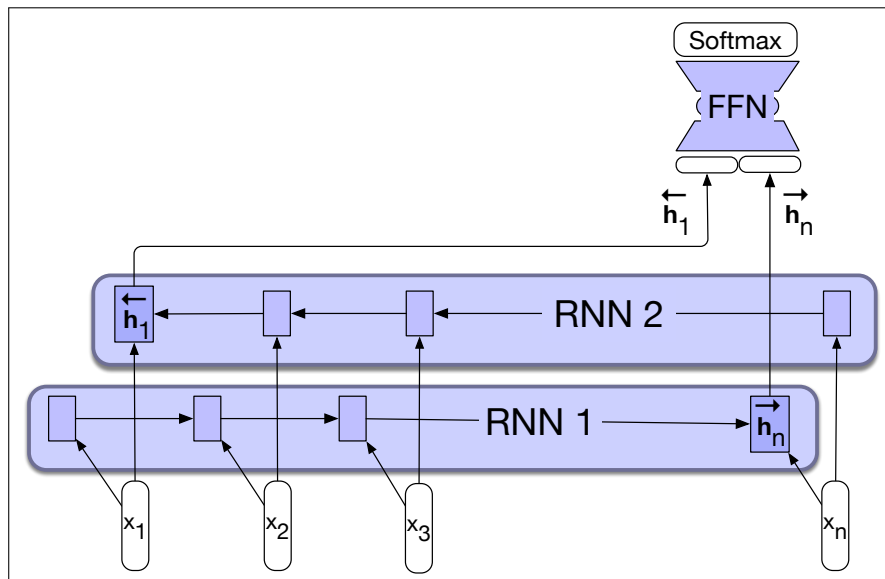


Figure 9.12 A bidirectional RNN for sequence classification. The final hidden units from the forward and backward passes are combined to represent the entire sequence. This combined representation serves as input to the subsequent classifier.

9.5 The LSTM

In practice, it is quite difficult to train RNNs for tasks that require a network to make use of information distant from the current point of processing. Despite having access to the entire preceding sequence, the information encoded in hidden states tends to be fairly local, more relevant to the most recent parts of the input sequence and recent decisions. Yet distant information is critical to many language applications. Consider the following example in the context of language modeling.

(9.19) The flights the airline was cancelling were full.

Assigning a high probability to *was* following *airline* is straightforward since *airline* provides a strong local context for the singular agreement. However, assigning an appropriate probability to *were* is quite difficult, not only because the plural *flights* is quite distant, but also because the intervening context involves singular constituents. Ideally, a network should be able to retain the distant information about plural *flights* until it is needed, while still processing the intermediate parts of the sequence correctly.

One reason for the inability of RNNs to carry forward critical information is that the hidden layers, and, by extension, the weights that determine the values in the hidden layer, are being asked to perform two tasks simultaneously: provide information useful for the current decision, and updating and carrying forward information required for future decisions.

A second difficulty with training RNNs arises from the need to backpropagate the error signal back through time. Recall from Section 9.1.2 that the hidden layer at time t contributes to the loss at the next time step since it takes part in that calculation. As a result, during the backward pass of training, the hidden layers are subject to repeated multiplications, as determined by the length of the sequence. A frequent result of this process is that the gradients are eventually driven to zero, a situation called the **vanishing gradients** problem.

vanishing
gradients

To address these issues, more complex network architectures have been designed to explicitly manage the task of maintaining relevant context over time, by enabling the network to learn to forget information that is no longer needed and to remember information required for decisions still to come.

The most commonly used such extension to RNNs is the **long short-term memory** (LSTM) network (Hochreiter and Schmidhuber, 1997). LSTMs divide the context management problem into two subproblems: removing information no longer needed from the context, and adding information likely to be needed for later decision making. The key to solving both problems is to learn how to manage this context rather than hard-coding a strategy into the architecture. LSTMs accomplish this by first adding an explicit context layer to the architecture (in addition to the usual recurrent hidden layer), and through the use of specialized neural units that make use of *gates* to control the flow of information into and out of the units that comprise the network layers. These gates are implemented through the use of additional weights that operate sequentially on the input, and previous hidden layer, and previous context layers.

long short-term
memory

The gates in an LSTM share a common design pattern; each consists of a feed-forward layer, followed by a sigmoid activation function, followed by a pointwise multiplication with the layer being gated. The choice of the sigmoid as the activation function arises from its tendency to push its outputs to either 0 or 1. Combining this with a pointwise multiplication has an effect similar to that of a binary mask. Values

in the layer being gated that align with values near 1 in the mask are passed through nearly unchanged; values corresponding to lower values are essentially erased.

forget gate

The first gate we'll consider is the **forget gate**. The purpose of this gate is to delete information from the context that is no longer needed. The forget gate computes a weighted sum of the previous state's hidden layer and the current input and passes that through a sigmoid. This mask is then multiplied element-wise by the context vector to remove the information from context that is no longer required. Element-wise multiplication of two vectors (represented by the operator $\odot$, and sometimes called the **Hadamard product**) is the vector of the same dimension as the two input vectors, where each element i is the product of element i in the two input vectors:

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t) \quad (9.20)$$

$$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t \quad (9.21)$$

The next task is to compute the actual information we need to extract from the previous hidden state and current inputs—the same basic computation we've been using for all our recurrent networks.

$$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t) \quad (9.22)$$

add gate

Next, we generate the mask for the **add gate** to select the information to add to the current context.

$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t) \quad (9.23)$$

$$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t \quad (9.24)$$

Next, we add this to the modified context vector to get our new context vector.

$$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t \quad (9.25)$$

output gate

The final gate we'll use is the **output gate** which is used to decide what information is required for the current hidden state (as opposed to what information needs to be preserved for future decisions).

$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t) \quad (9.26)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \quad (9.27)$$

Fig. 9.13 illustrates the complete computation for a single LSTM unit. Given the appropriate weights for the various gates, an LSTM accepts as input the context layer, and hidden layer from the previous time step, along with the current input vector. It then generates updated context and hidden vectors as output. The hidden layer, $\mathbf{h}_t$, can be used as input to subsequent layers in a stacked RNN, or to generate an output for the final layer of a network.

9.5.1 Gated Units, Layers and Networks

The neural units used in LSTMs are obviously much more complex than those used in basic feedforward networks. Fortunately, this complexity is encapsulated within the basic processing units, allowing us to maintain modularity and to easily experiment with different architectures. To see this, consider Fig. 9.14 which illustrates the inputs and outputs associated with each kind of unit.

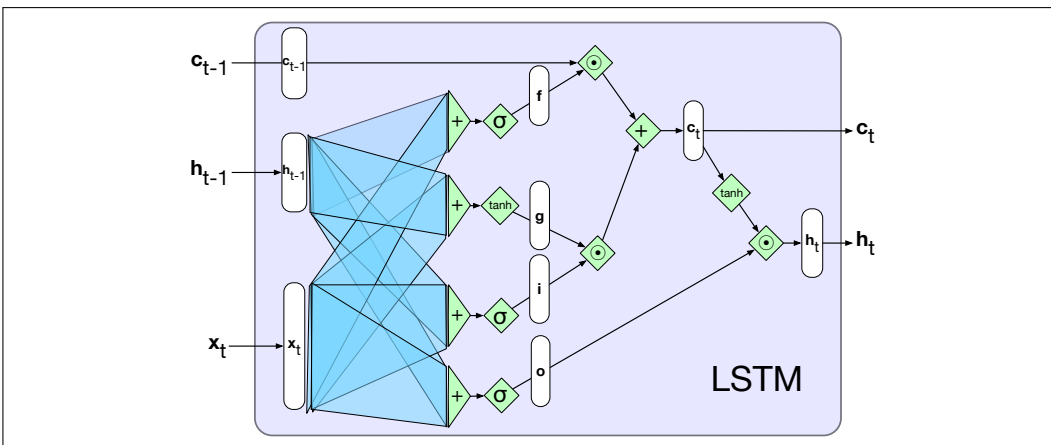


Figure 9.13 A single LSTM unit displayed as a computation graph. The inputs to each unit consists of the current input, x , the previous hidden state, h_{t-1} , and the previous context, c_{t-1} . The outputs are a new hidden state, h_t and an updated context, c_t .

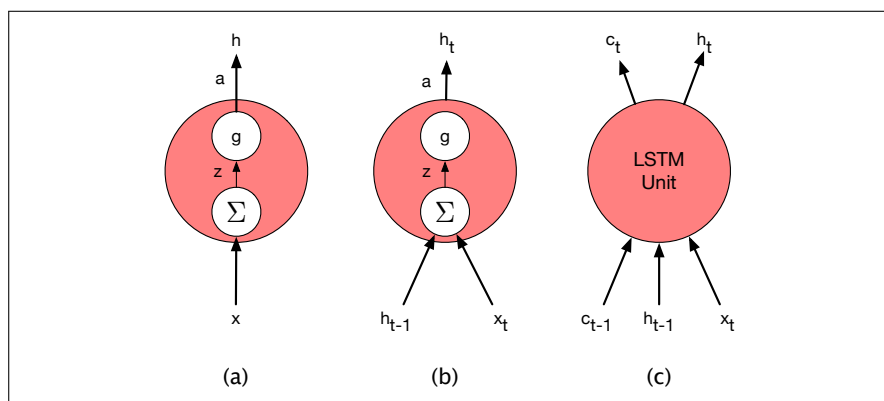


Figure 9.14 Basic neural units used in feedforward, simple recurrent networks (SRN), and long short-term memory (LSTM).

At the far left, (a) is the basic feedforward unit where a single set of weights and a single activation function determine its output, and when arranged in a layer there are no connections among the units in the layer. Next, (b) represents the unit in a simple recurrent network. Now there are two inputs and an additional set of weights to go with it. However, there is still a single activation function and output.

The increased complexity of the LSTM units is encapsulated within the unit itself. The only additional external complexity for the LSTM over the basic recurrent unit (b) is the presence of the additional context vector as an input and output.

This modularity is key to the power and widespread applicability of LSTM units. LSTM units (or other varieties, like GRUs) can be substituted into any of the network architectures described in Section 9.4. And, as with simple RNNs, multi-layered networks making use of gated units can be unrolled into deep feedforward networks and trained in the usual fashion with backpropagation. In practice, therefore, LSTMs rather than RNNs have become the standard unit for any modern system that makes use of recurrent networks.

9.6 Summary: Common RNN NLP Architectures

We've now introduced the RNN, seen advanced components like stacking multiple layers and using the LSTM version, and seen how the RNN can be applied to various tasks. Let's take a moment to summarize the architectures for these applications.

Fig. 9.15 shows the three architectures we've discussed so far: sequence labeling, sequence classification, and language modeling. In sequence labeling (for example for part of speech tagging), we train a model to produce a label for each input word or token. In sequence classification, for example for sentiment analysis, we ignore the output for each token, and only take the value from the end of the sequence (and similarly the model's training signal comes from backpropagation from that last token). In language modeling, we train the model to predict the next word at each token step. In the next section we'll introduce a fourth architecture, the **encoder-decoder**.

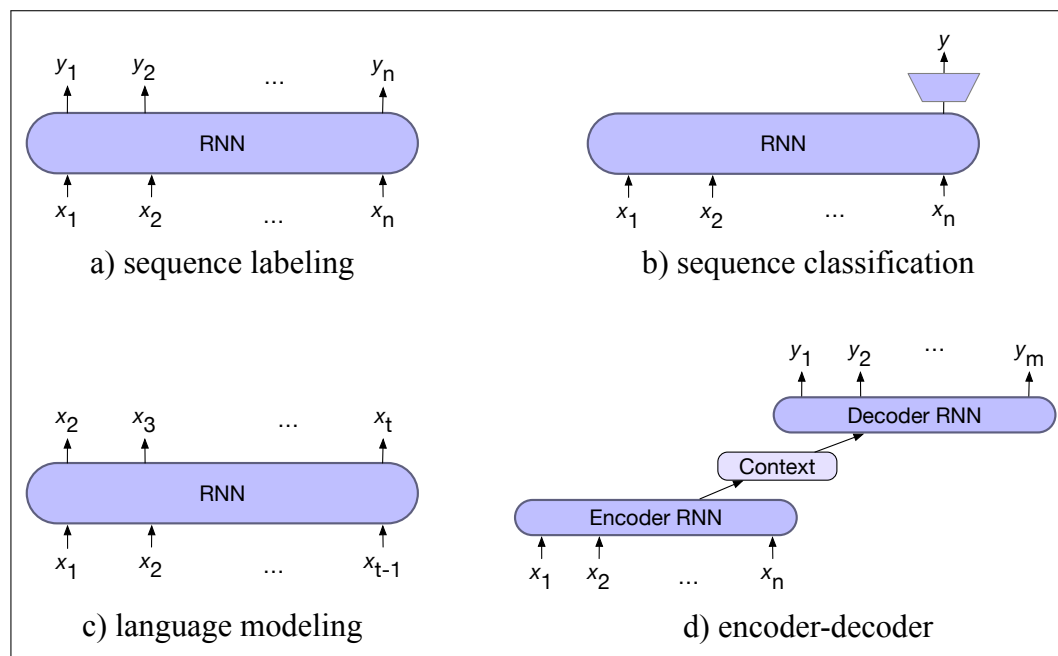


Figure 9.15 Four architectures for NLP tasks. In sequence labeling (POS or named entity tagging) we map each input token x_i to an output token y_i . In sequence classification we map the entire input sequence to a single class. In language modeling we output the next token conditioned on previous tokens. In the encoder model we have two separate RNN models, one of which maps from an input sequence $\mathbf{x}$ to an intermediate representation we call the **context**, and a second of which maps from the context to an output sequence $\mathbf{y}$.

9.7 The Encoder-Decoder Model with RNNs

In this section we introduce a new model, the encoder-decoder model, which is used when we are taking an input sequence and translating it to an output sequence that is of a different length than the input, and doesn't align with it in a word-to-word way. Recall that in the sequence labeling task, we have two sequences, but they are

the same length (for example in part-of-speech tagging each token gets an associated tag), each input is associated with a specific output, and the labeling for that output takes mostly local information. Thus deciding whether a word is a verb or a noun, we look mostly at the word and the neighboring words.

By contrast, encoder-decoder models are used especially for tasks like machine translation, where the input sequence and output sequence can have different lengths and the mapping between a token in the input and a token in the output can be very indirect (in some languages the verb appears at the beginning of the sentence; in other languages at the end). We'll introduce machine translation in detail in Chapter 13, but for now we'll just point out that the mapping for a sentence in English to a sentence in Tagalog or Yoruba can have very different numbers of words, and the words can be in a very different order.

encoder-decoder

Encoder-decoder networks, sometimes called **sequence-to-sequence** networks, are models capable of generating contextually appropriate, arbitrary length, output sequences given an input sequence. Encoder-decoder networks have been applied to a very wide range of applications including summarization, question answering, and dialogue, but they are particularly popular for machine translation.

The key idea underlying these networks is the use of an **encoder** network that takes an input sequence and creates a contextualized representation of it, often called the **context**. This representation is then passed to a **decoder** which generates a task-specific output sequence. Fig. 9.16 illustrates the architecture

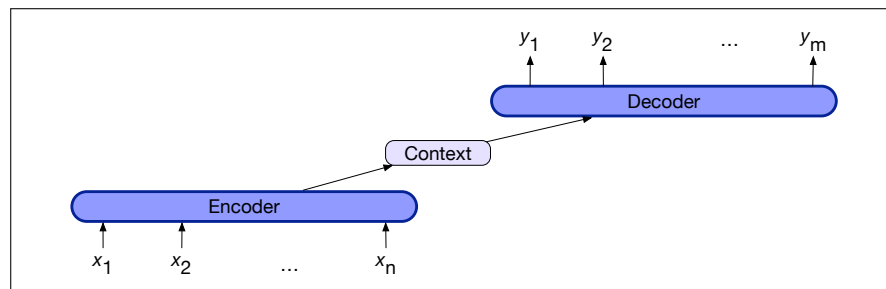


Figure 9.16 The encoder-decoder architecture. The context is a function of the hidden representations of the input, and may be used by the decoder in a variety of ways.

Encoder-decoder networks consist of three components:

1. An **encoder** that accepts an input sequence, x_1^n , and generates a corresponding sequence of contextualized representations, h_1^n . LSTMs, convolutional networks, and Transformers can all be employed as encoders.
2. A **context vector**, c , which is a function of h_1^n , and conveys the essence of the input to the decoder.
3. A **decoder**, which accepts c as input and generates an arbitrary length sequence of hidden states h_1^m , from which a corresponding sequence of output states y_1^m , can be obtained. Just as with encoders, decoders can be realized by any kind of sequence architecture.

In this section we'll describe an encoder-decoder network based on a pair of RNNs, but we'll see in Chapter 13 how to apply them to transformers as well. We'll build up the equations for encoder-decode models by starting with the conditional RNN language model $p(y)$, the probability of a sequence y .

Recall that in any language model, we can break down the probability as follows:

$$p(y) = p(y_1)p(y_2|y_1)p(y_3|y_1, y_2) \dots P(y_m|y_1, \dots, y_{m-1}) \quad (9.28)$$

In RNN language modeling, at a particular time t , we pass the prefix of $t - 1$ tokens through the language model, using forward inference to produce a sequence of hidden states, ending with the hidden state corresponding to the last word of the prefix. We then use the final hidden state of the prefix as our starting point to generate the next token.

More formally, if g is an activation function like $\tanh$ or ReLU, a function of the input at time t and the hidden state at time $t - 1$, and f is a softmax over the set of possible vocabulary items, then at time t the output $\mathbf{y}_t$ and hidden state $\mathbf{h}_t$ are computed as:

$$\mathbf{h}_t = g(\mathbf{h}_{t-1}, \mathbf{x}_t) \quad (9.29)$$

$$\mathbf{y}_t = f(\mathbf{h}_t) \quad (9.30)$$

sentence
separation

We only have to make one slight change to turn this language model with autoregressive generation into an encoder-decoder model that is a translation model that can translate from a **source text** in one language to a **target text** in a second: add a **sentence separation** marker at the end of the source text, and then simply concatenate the target text.

Let's use $\langle s \rangle$ for our sentence separate token, and let's think about translating an English source text ("the green witch arrived"), to a Spanish sentence ("*llegó la bruja verde*") (which can be glossed word-by-word as 'arrived the witch green'). We could also illustrate encoder-decoder models with a question-answer pair, or a text-summarization pair, but m

Let's use x to refer to the source text (in this case in English) plus the separator token $\langle s \rangle$, and y to refer to the target text y (in this case in Spanish). Then an encoder-decoder model computes the probability $p(y|x)$ as follows:

$$p(y|x) = p(y_1|x)p(y_2|y_1,x)p(y_3|y_1,y_2,x)\dots P(y_m|y_1,\dots,y_{m-1},x) \quad (9.31)$$

Fig. 9.17 shows the setup for a simplified version of the encoder-decoder model (we'll see the full model, which requires the new concept of **attention**, in the next section).

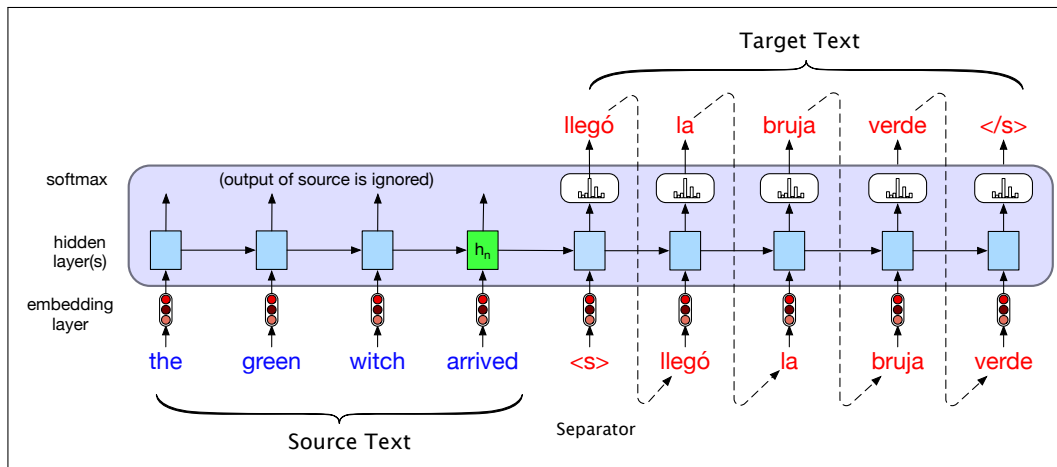


Figure 9.17 Translating a single sentence (inference time) in the basic RNN version of encoder-decoder approach to machine translation. Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

Fig. 9.17 shows an English source text ("the green witch arrived"), a sentence separator token ($\langle s \rangle$, and a Spanish target text ("*llegó la bruja verde*"). To trans-

late a source text, we run it through the network performing forward inference to generate hidden states until we get to the end of the source. Then we begin autoregressive generation, asking for a word in the context of the hidden layer from the end of the source input as well as the end-of-sentence marker. Subsequent words are conditioned on the previous hidden state and the embedding for the last word generated.

Let's formalize and generalize this model a bit in Fig. 9.18. (To help keep things straight, we'll use the superscripts e and d where needed to distinguish the hidden states of the encoder and the decoder.) The elements of the network on the left process the input sequence x and comprise the **encoder**. While our simplified figure shows only a single network layer for the encoder, stacked architectures are the norm, where the output states from the top layer of the stack are taken as the final representation. A widely used encoder design makes use of stacked biLSTMs where the hidden states from top layers from the forward and backward passes are concatenated as described in Chapter 9 to provide the contextualized representations for each time step.

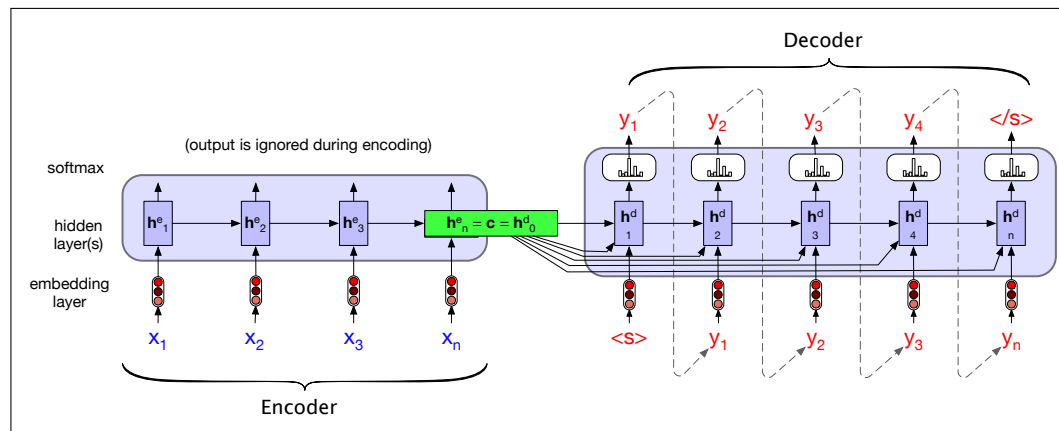


Figure 9.18 A more formal version of translating a sentence at inference time in the basic RNN-based encoder-decoder architecture. The final hidden state of the encoder RNN, h_n^e , serves as the context for the decoder in its role as h_0^d in the decoder RNN.

The entire purpose of the encoder is to generate a contextualized representation of the input. This representation is embodied in the final hidden state of the encoder, h_n^e . This representation, also called **c** for **context**, is then passed to the decoder.

The **decoder** network on the right takes this state and uses it to initialize the first hidden state of the decoder. That is, the first decoder RNN cell uses c as its prior hidden state h_0^d . The decoder autoregressively generates a sequence of outputs, an element at a time, until an end-of-sequence marker is generated. Each hidden state is conditioned on the previous hidden state and the output generated in the previous state.

One weakness of this approach as described so far is that the influence of the context vector, c , will wane as the output sequence is generated. A solution is to make the context vector c available at each step in the decoding process by adding it as a parameter to the computation of the current hidden state, using the following equation (illustrated in Fig. 9.19):

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \quad (9.32)$$

Now we're ready to see the full equations for this version of the decoder in the basic

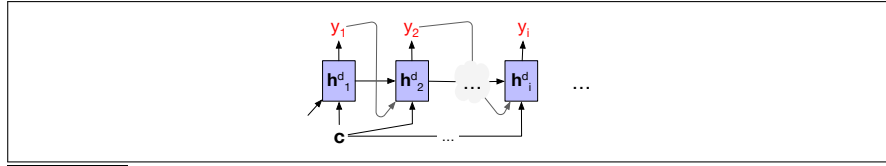


Figure 9.19 Allowing every hidden state of the decoder (not just the first decoder state) to be influenced by the context c produced by the encoder.

encoder-decoder model, with context available at each decoding timestep. Recall that g is a stand-in for some flavor of RNN and $\hat{y}_{t-1}$ is the embedding for the output sampled from the softmax at the previous step:

$$\begin{aligned}
 c &= h_n^e \\
 h_0^d &= c \\
 h_t^d &= g(\hat{y}_{t-1}, h_{t-1}^d, c) \\
 z_t &= f(h_t^d) \\
 y_t &= \text{softmax}(z_t)
 \end{aligned} \tag{9.33}$$

Finally, as shown earlier, the output y at each time step consists of a softmax computation over the set of possible outputs (the vocabulary, in the case of language modeling or MT). We compute the most likely output at each time step by taking the argmax over the softmax output:

$$\hat{y}_t = \text{argmax}_{w \in V} P(w|x, y_1 \dots y_{t-1}) \tag{9.34}$$

9.7.1 Training the Encoder-Decoder Model

Encoder-decoder architectures are trained end-to-end, just as with the RNN language models of Chapter 9. Each training example is a tuple of paired strings, a source and a target. Concatenated with a separator token, these source-target pairs can now serve as training data.

For MT, the training data typically consists of sets of sentences and their translations. These can be drawn from standard datasets of aligned sentence pairs, as we'll discuss in Section 13.2.2. Once we have a training set, the training itself proceeds as with any RNN-based language model. The network is given the source text and then starting with the separator token is trained autoregressively to predict the next word, as shown in Fig. 9.20.

Note the differences between training (Fig. 9.20) and inference (Fig. 9.17) with respect to the outputs at each time step. The decoder during inference uses its own estimated output $\hat{y}_t$ as the input for the next time step x_{t+1} . Thus the decoder will tend to deviate more and more from the gold target sentence as it keeps generating more tokens. In training, therefore, it is more common to use **teacher forcing** in the decoder. Teacher forcing means that we force the system to use the gold target token from training as the next input x_{t+1} , rather than allowing it to rely on the (possibly erroneous) decoder output $\hat{y}_t$. This speeds up training.

teacher forcing

9.8 Attention

The simplicity of the encoder-decoder model is its clean separation of the encoder—which builds a representation of the source text—from the decoder, which uses this

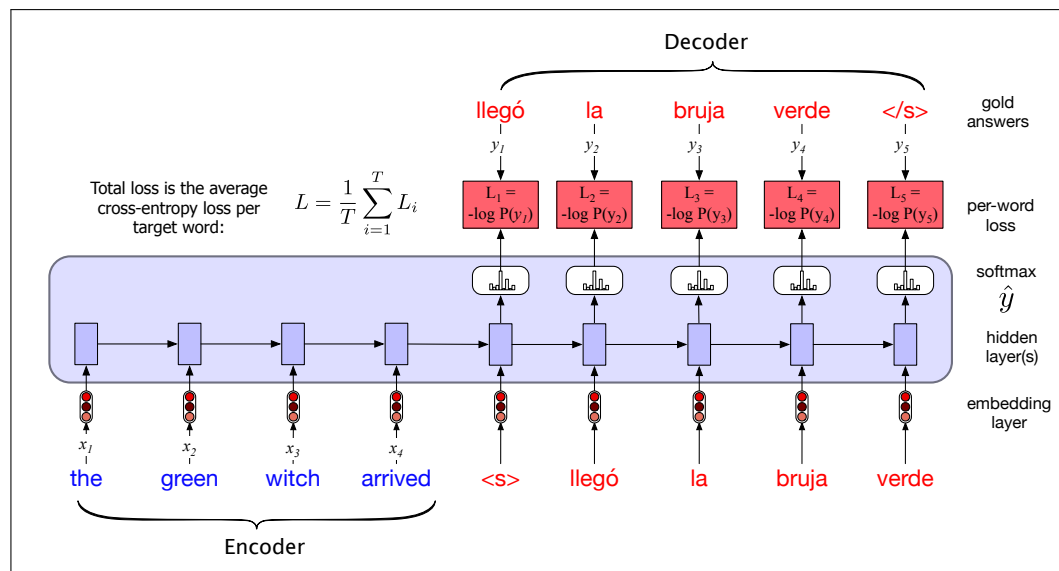


Figure 9.20 Training the basic RNN encoder-decoder approach to machine translation. Note that in the decoder we usually don't propagate the model's softmax outputs $\hat{y}_t$, but use **teacher forcing** to force each input to the correct gold value for training. We compute the softmax output distribution over $\hat{y}$ in the decoder in order to compute the loss at each token, which can then be averaged to compute a loss for the sentence.

context to generate a target text. In the model as we've described it so far, this context vector is h_n , the hidden state of the last (nth) time step of the source text. This final hidden state is thus acting as a **bottleneck**: it must represent absolutely everything about the meaning of the source text, since the only thing the decoder knows about the source text is what's in this context vector (Fig. 9.21). Information at the beginning of the sentence, especially for long sentences, may not be equally well represented in the context vector.

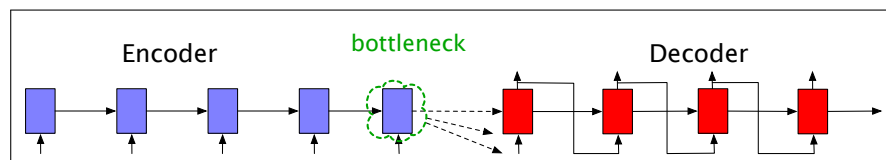


Figure 9.21 Requiring the context c to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.

attention mechanism

The **attention mechanism** is a solution to the bottleneck problem, a way of allowing the decoder to get information from *all* the hidden states of the encoder, not just the last hidden state.

In the attention mechanism, as in the vanilla encoder-decoder model, the context vector $\mathbf{c}$ is a single vector that is a function of the hidden states of the encoder, that is, $\mathbf{c} = f(\mathbf{h}_1^e \dots \mathbf{h}_n^e)$. Because the number of hidden states varies with the size of the input, we can't use the entire tensor of encoder hidden state vectors directly as the context for the decoder.

The idea of attention is instead to create the single fixed-length vector c by taking a weighted sum of all the encoder hidden states. The weights focus on ('attend to') a particular part of the source text that is relevant for the token the decoder is currently producing. Attention thus replaces the static context vector with one that is dynamically derived from the encoder hidden states, different for each token in

decoding.

This context vector, $\mathbf{c}_i$, is generated anew with each decoding step i and takes all of the encoder hidden states into account in its derivation. We then make this context available during decoding by conditioning the computation of the current decoder hidden state on it (along with the prior hidden state and the previous output generated by the decoder), as we see in this equation (and Fig. 9.22):

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i) \quad (9.35)$$

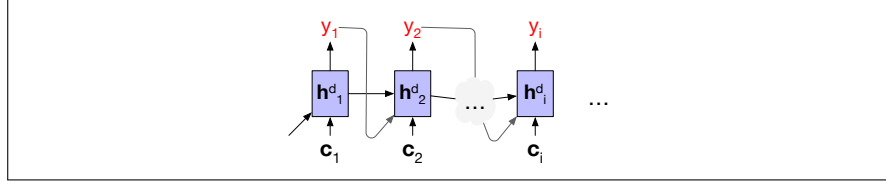


Figure 9.22 The attention mechanism allows each hidden state of the decoder to see a different, dynamic, context, which is a function of all the encoder hidden states.

The first step in computing $\mathbf{c}_i$ is to compute how much to focus on each encoder state, how *relevant* each encoder state is to the decoder state captured in $\mathbf{h}_{i-1}^d$. We capture relevance by computing—at each state i during decoding—a *score*($\mathbf{h}_{i-1}^d, \mathbf{h}_j^e$) for each encoder state j .

dot-product
attention

The simplest such score, called **dot-product attention**, implements relevance as similarity: measuring how similar the decoder hidden state is to an encoder hidden state, by computing the dot product between them:

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e \quad (9.36)$$

The score that results from this dot product is a scalar that reflects the degree of similarity between the two vectors. The vector of these scores across all the encoder hidden states gives us the relevance of each encoder state to the current step of the decoder.

To make use of these scores, we'll normalize them with a softmax to create a vector of weights, α_{ij} , that tells us the proportional relevance of each encoder hidden state j to the prior hidden decoder state, $\mathbf{h}_{i-1}^d$.

$$\begin{aligned} \alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) \quad \forall j \in e) \\ &= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))} \end{aligned} \quad (9.37)$$

Finally, given the distribution in α , we can compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states.

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e \quad (9.38)$$

With this, we finally have a fixed-length context vector that takes into account information from the entire encoder state that is dynamically updated to reflect the needs of the decoder at each step of decoding. Fig. 9.23 illustrates an encoder-decoder network with attention, focusing on the computation of one context vector $\mathbf{c}_i$.

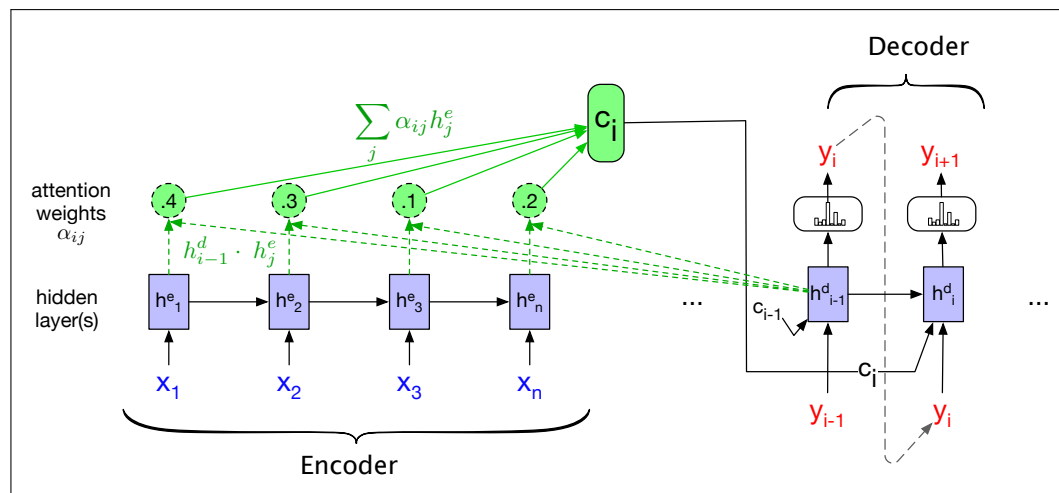


Figure 9.23 A sketch of the encoder-decoder network with attention, focusing on the computation of c_i . The context value c_i is one of the inputs to the computation of h_i^d . It is computed by taking the weighted sum of all the encoder hidden states, each weighted by their dot product with the prior decoder hidden state h_{i-1}^d .

It's also possible to create more sophisticated scoring functions for attention models. Instead of simple dot product attention, we can get a more powerful function that computes the relevance of each encoder hidden state to the decoder hidden state by parameterizing the score with its own set of weights, $\mathbf{W}_s$.

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \mathbf{W}_s \mathbf{h}_j^e$$

The weights $\mathbf{W}_s$, which are then trained during normal end-to-end training, give the network the ability to learn which aspects of similarity between the decoder and encoder states are important to the current application. This bilinear model also allows the encoder and decoder to use different dimensional vectors, whereas the simple dot-product attention requires that the encoder and decoder hidden states have the same dimensionality.

We'll return to the concept of attention when we defined the transformer architecture in Chapter 10, which is based on a slight modification of attention called **self-attention**.

9.9 Summary

This chapter has introduced the concepts of recurrent neural networks and how they can be applied to language problems. Here's a summary of the main points that we covered:

- In simple Recurrent Neural Networks sequences are processed one element at a time, with the output of each neural unit at time t based both on the current input at t and the hidden layer from time $t - 1$.
- RNNs can be trained with a straightforward extension of the backpropagation algorithm, known as **backpropagation through time** (BPTT).
- Simple recurrent networks fail on long inputs because of problems like **vanishing gradients**; instead modern systems use more complex gated architec-